CS 530 Lab Notebook

Lab 6

Name: Jarren Calizo
OdinID: calizo
Date: November 5, 2025

Table of Contents

Lab Overview 4
Lab 6.1a EB Guestbook 5
Step 1. Elastic Beanstalk Overview 5
Step 2. Deploying the Sample Application 5
Step 3. Running and Verifying the Application 6
1. Take a screenshot showing it has been brought up successfully 6
Step 4. Handling Failures Seamlessly 8
1. Take a screenshot of the replacement VM being started. 8
Step 5. Clean Up 9
Step 6. Elastic Beanstalk Guestbook 10
Step 7. Deploying the Guestbook 11
1. Take a screenshot of the Guestbook including the URL with the entry in it. 12
2. Take a screenshot of them. 13
Step 8. Clean Up 13
Lab 6.1g App Engine Guestbook 15
Step 1. App Engine 15
Step 2. app.yaml 15
Step 3. Deploying the Guestbook 15
1. Take a screenshot of the output that includes the URL in the address bar for your lab notebook. 16
Step 4. Handling failures seamlessly 17
1. Take a screenshot of them. 17
Step 5. Clean Up 18
Lab 6.2g Cloud Run, Secret Manager (Web proxy) 20
Step 1. Cloud Run 20
Step 2. Application Setup 20
Step 3. app.py 20
Step 4. templates/proxy.html 21
Step 5. requirements.txt 21
Step 6. Dockerfile 21
Step 7. Build and Local Test in Cloud Shell 22
Step 8. Setup secret proxy 23
1. Take a screenshot of the proxy and its results including the URL containing your OdinID 23
2. What is the security advantage of passing in the secret proxy route as an environment variable? 23

Step 9. Artifact Registry 25
Step 10. Cloud Build 25
1. Take a screenshot of the image in the registry that shows the size of the container for your lab notebook. 26
Step 11. Deploy to Cloud Run 27
1. Take a screenshot of it that includes the proxy URL for your lab notebook. 27
2. Take a screenshot of the error page that includes the proxy URL for your lab notebook. 28
Step 12. Secret Manager 29
Step 13. Deploy to Cloud Run with Secret Manager 30
1. Take a screenshot of it that includes the proxy URL for your lab notebook. 31
2. Identify the vulnerability in your lab notebook that Google has prevented. 31
Step 14. Clean Up 31
Lab 6.3a ECS Guestbook 33
Step 1. Prepare a container image 33
Step 2. ECS overview 33
Step 3. ECS task definition 34
Step 4. Create a Cluster and a Service 34
Step 5. Examine the service 35
1. Take a screenshot of the DNS name of the guestbook-lb load balancer for your lab notebook 35
Step 6. Visit the site 36
1. Take a screenshot of the Guestbook app running in a browser that includes the DNS name of the site. 36
Step 7. Clean Up 36
Lab 6.3g Cloud Run Guestbook 38
Step 1. Cloud Run 38
Step 2. Prepare a container image 38
1. Take a screenshot that includes the output of the command and the time it took to execute. 39
2. Take a screenshot showing the container image and its virtual size 40
Step 3. Deploy container with minimal privileges 40
Step 4. View the Guestbook 41
1. Take a screenshot that includes the URL Cloud Run has created for your site. 41
2. What port do container instances listen on? 42

3. What are the maximum number of instances Cloud Run will autoscale up to for your service? 42
Step 5. Clean Up 42
Lab 6.4g Cloud Functions, PubSub 44
Step 1. Cloud Functions image blurring 44
Step 2. Services setup 44
Step 3. Code 44
Step 4. Code (2) 44
1. After downloading the file from the bucket, where is it stored? 44
2. What class in the ImageMagick package is used to do the blurring of the file? 45
3. What lines of code perform the blurring of the image and its storage back into the filesystem? 45
Step 5. Set up service account 45
Step 6. Deploy the function 45
Step 7. Test function 46
1. Take a screenshot of the blurred image in the output bucket for your lab notebook 46
2. Include a screenshot of the output logs that show that the above image was blurred. 47
Step 8. Clean up 47
Step 9. Pub/Sub 48
Step 10. Create VM for subscriber 48
Step 11. PubSub via CLI 48
1. Why are there no items returned? 49
Step 12. PubSub via CLI (2) 49
1. What is the messageId of the published message? 49
2. Take a screenshot of the output of the successful pull that includes the message and its messageId. 50
Step 13-14. Pub/Sub via Python 50
Step 15. Test programs and clean up 51
1. Take a screenshot showing the messageIds and messages sent 51
2. Take a screenshot showing the same messageIds and messages received 52

Lab Overview

Lab 6 embodied the essence of cloud computing’s orchestration and automation through a comparative exploration of AWS and Google Cloud. Across its five sub-labs, the experience bridged traditional PaaS and modern serverless paradigms, illustrating how web applications evolve from managed environments to fully event-driven architectures.

It began with Elastic Beanstalk, AWS’s managed platform that abstracts infrastructure behind an automated orchestration layer. Deploying the Guestbook here highlighted elasticity through load balancing and automatic fault recovery - EC2 instances spun up and healed seamlessly without manual intervention. This reaffirmed a central tenet of cloud design: reliability through automation, not redundancy through effort.

The transition to Google App Engine mirrored this philosophy under a different ecosystem. By deploying the same Guestbook application through a simple app.yaml configuration, App Engine demonstrated how developers can declaratively define compute and scaling logic. Watching instances self-replace after manual termination underscored Google’s automated resilience and fault-tolerance ethos - a theme that quietly echoed through the entire lab.

The next section, Cloud Run and Secret Manager, deepened the concept of configuration isolation and security. Building a Flask-based proxy application and encapsulating it within a Docker container represented a move toward flexible containerized deployment. Passing secret routes as environment variables - and later integrating Secret Manager - reinforced secure DevOps practices: separating secrets from code and enforcing runtime-only visibility. This was cloud security and CI/CD philosophy converging in real time, a moment where understanding met responsibility.

Shifting to ECS (Elastic Container Service) and Fargate brought the AWS counterpart into focus. The Guestbook’s container image was orchestrated through ECS services and load-balanced by an Application Load Balancer, representing the DevOps pipeline in its truest form - build, push, deploy, verify, and tear down. Every automation reflected operational maturity: the kind enterprises rely on for scaling production workloads with precision.

Finally, Cloud Functions and Pub/Sub closed the journey with an event-driven crescendo. The function that blurred offensive images tied together machine learning (Vision API), storage triggers, and Pub/Sub messaging to showcase the modular beauty of serverless workflows. Each message exchange between publisher and subscriber - from Cloud Shell to VM - revealed how asynchronous communication can stitch independent services into one intelligent pipeline.

Across all sections, Lab 6 was more than a technical exercise; it was a story of abstraction and control - of learning when to let go of infrastructure and when to architect around automation. It revealed that true mastery of the cloud lies not in managing servers, but in designing resilient systems that thrive without them. It was a lesson in both humility and empowerment - one line of YAML, one Dockerfile, and one function at a time.

Lab 6.1a EB Guestbook

Step 1. Elastic Beanstalk Overview

Elastic Beanstalk (EB) represents AWS's Platform-as-a-Service offering, designed to abstract away infrastructure concerns so developers can focus entirely on application logic. Unlike container-based deployment on Elastic Container Service where you define both the environment and the app, EB provides a preconfigured environment (such as Python, NodeJS, or Ruby) and automatically manages load balancing, scaling, and monitoring through CloudFormation. It's an elegant way to experience serverless elasticity without having to touch EC2 networking or IAM policy scripts directly.

This step served as a conceptual grounding, showing how AWS automates orchestration tasks that, in traditional setups, would require manually configuring VPCs, subnets, and auto-scaling groups.

Step 2. Deploying the Sample Application

Using the AWS Management Console, I navigated to Elastic Beanstalk and created a new application named eb-hello under the Python platform. Following the lab directions, I selected the Sample application for code deployment and used the High availability preset, which automatically provisions a load balancer across multiple availability zones for fault tolerance.

Under permissions, I assigned LabRole as the service role and LabInstanceProfile for the EC2 instance profile to ensure proper IAM access boundaries. Networking was configured to use the default VPC, with public IPs enabled and all subnets in us-east-1 selected for the load balancer. The database configuration step was intentionally skipped to maintain focus on application hosting rather than data persistence.

The storage and compute configuration used General Purpose SSD (gp3) as the root volume type, paired with the default security group. For auto-scaling, the environment was set to maintain a minimum of two instances, allowing Elastic Beanstalk to evenly distribute requests across them. Multiple t2 instance types were listed to grant flexibility during instance provisioning.

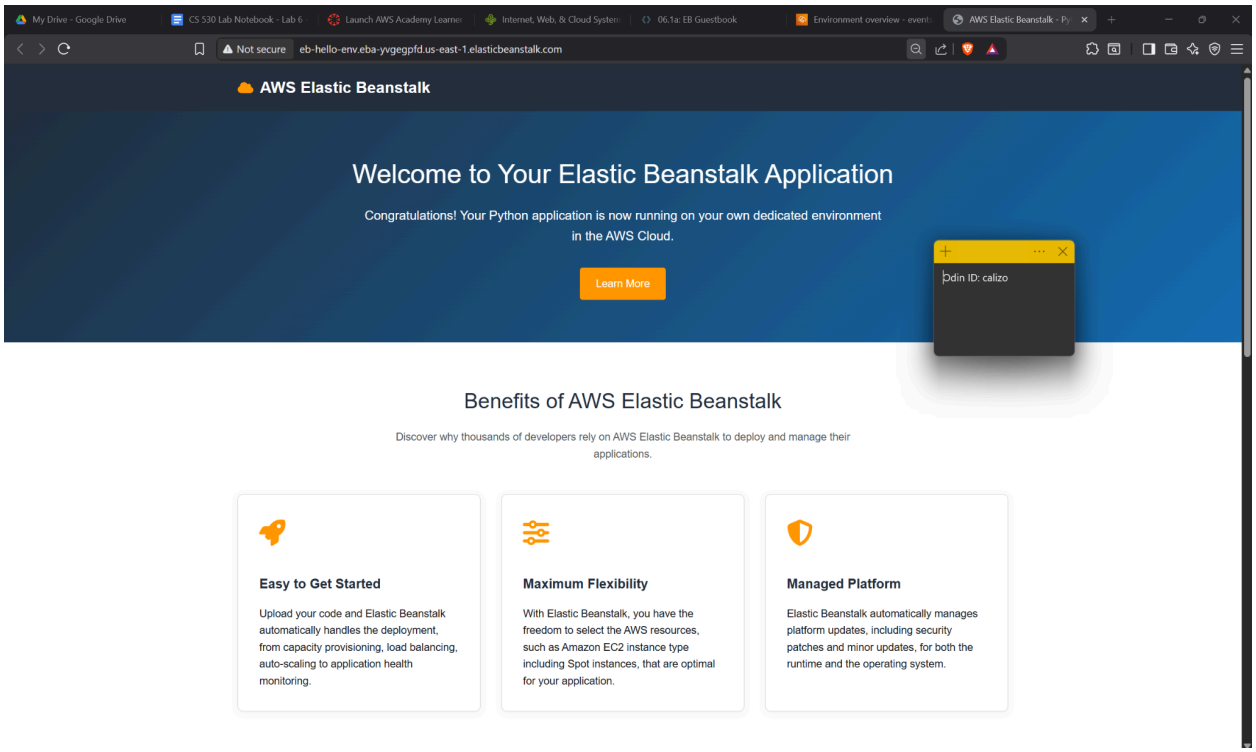
Once submitted, Elastic Beanstalk began its automated setup - launching EC2 instances, creating an Auto Scaling Group, assigning IAM roles, and setting up monitoring through CloudWatch - all orchestrated under the hood by CloudFormation. The process completed in approximately five minutes.

Step 3. Running and Verifying the Application

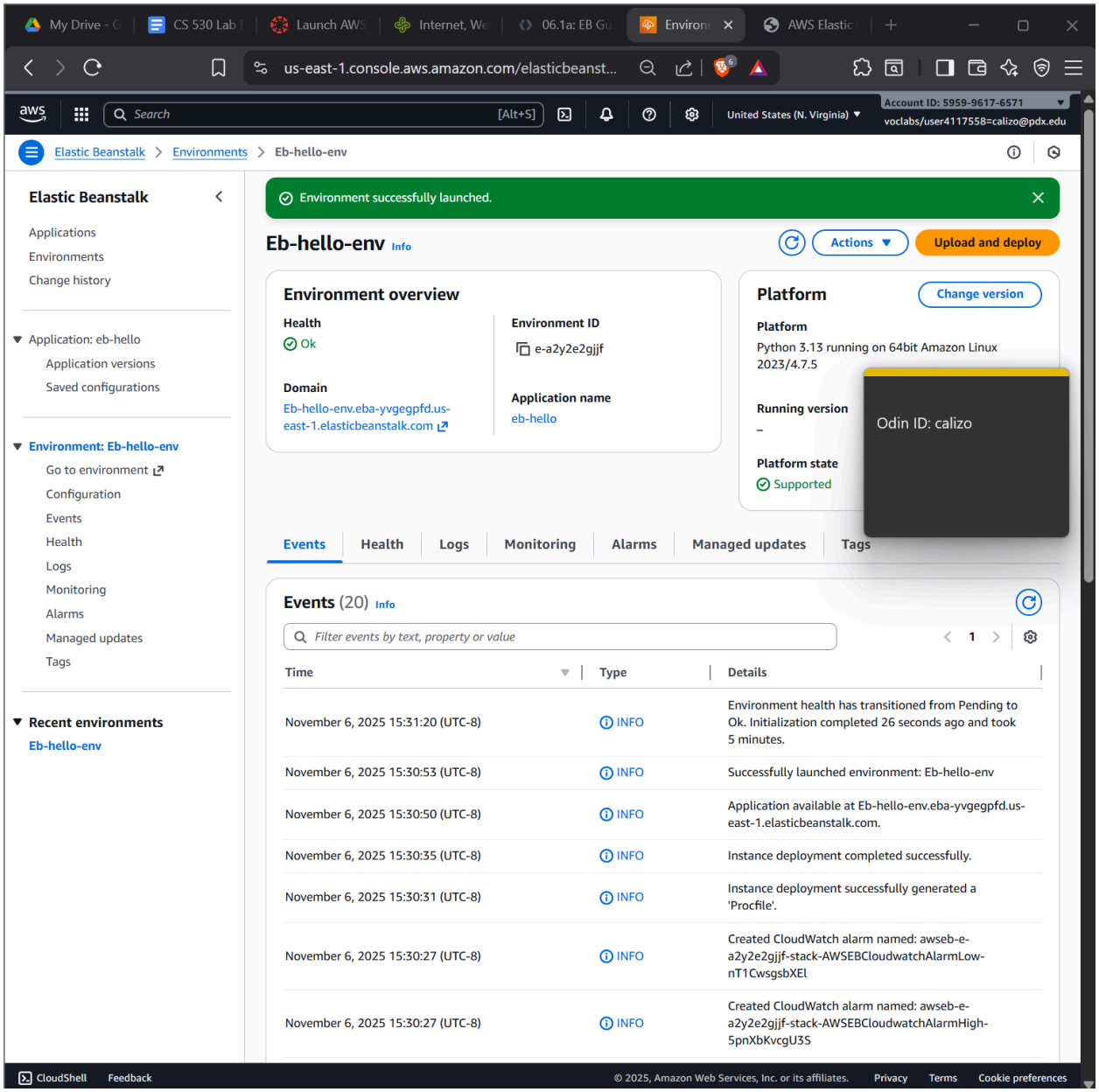
Upon successful deployment, the environment logs confirmed the message “Environment successfully launched,” with the health status marked OK. The domain link provided was:

- http://eb-hello-env.eba-yvgegpfd.us-east-1.elasticbeanstalk.com

1. Take a screenshot showing it has been brought up successfully



Opening this link in a browser displayed the Elastic Beanstalk Python sample application with the message “Congratulations! Your Python application is now running on your own dedicated environment in the AWS Cloud.” The screenshot below captured the result, verifying that the EB environment was active and reachable.



In the AWS console, the environment dashboard confirmed a green health indicator, correct application name (eb-hello), and Platform: Python 3.13 running on 64bit Amazon Linux 2023/4.7.5. The Events tab recorded CloudFormation’s sequence of actions, including instance creation, load balancer registration, and health status transitions from Pending to Ok.

Together, these screenshots serve as evidence of a fully functional deployment where AWS handled the orchestration end-to-end.

Insight

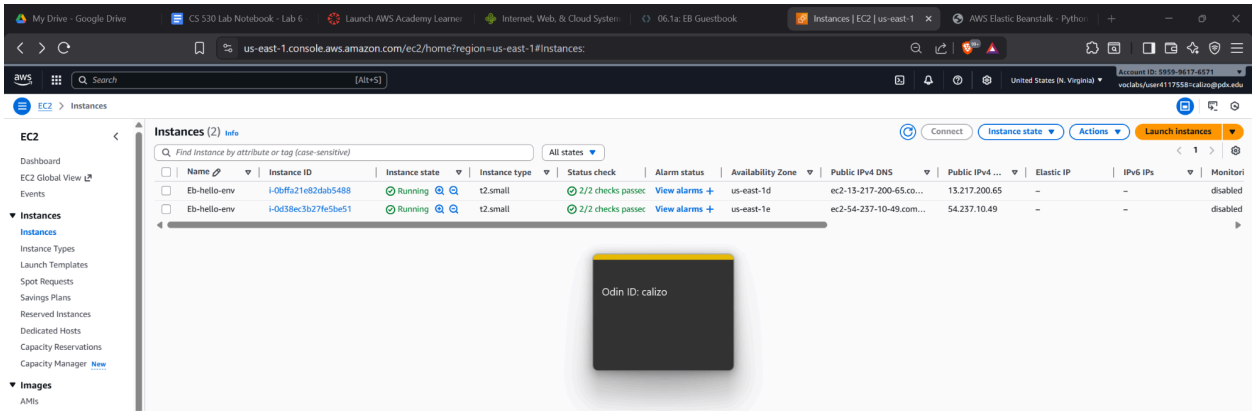
This initial Elastic Beanstalk deployment highlights the power of abstraction within cloud platforms. By simply defining configuration parameters, developers can create a fault-tolerant, monitored, and auto-scaling web service in minutes. It reinforces a foundational concept in cloud systems: effective architecture doesn’t always require low-level control - it often thrives on automation done right.

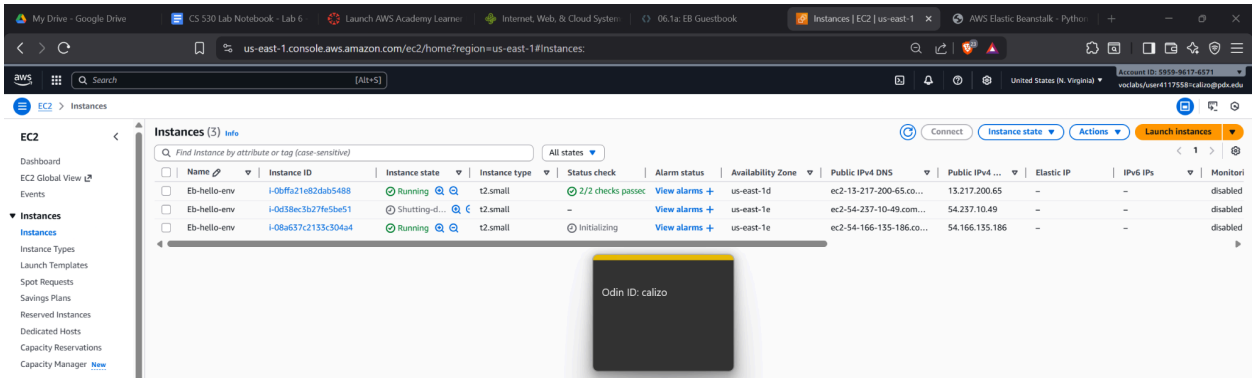
Step 4. Handling Failures Seamlessly

Elastic Beanstalk’s true strength lies in its ability to recover gracefully from failure. To demonstrate this, I navigated to the EC2 console and confirmed that two instances had been provisioned across separate availability zones (us-east-1d and us-east-1e) under the eb-hello-env environment. Both instances were healthy and running, as seen in the initial EC2 view.

To simulate a fault scenario, I manually terminated one of the running instances. Immediately, the EC2 dashboard reflected the instance transitioning to shutting-down status. Within minutes, Elastic Beanstalk detected the reduced instance count - falling below the defined minimum of two - and automatically launched a replacement instance to maintain availability and load balance integrity. The replacement appeared in the EC2 dashboard with a new instance ID and an initializing status before stabilizing to running.

1. Take a screenshot of the replacement VM being started.



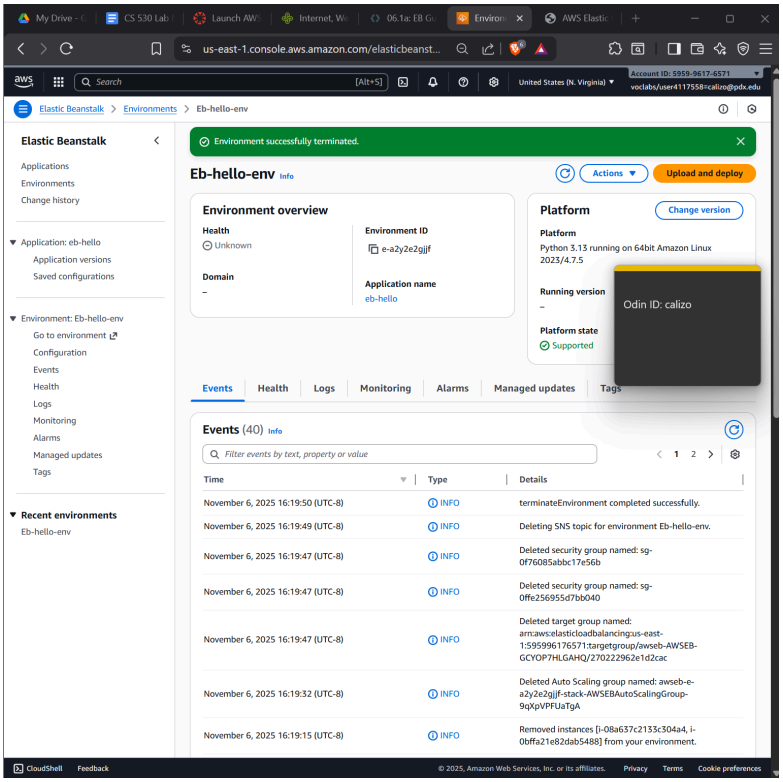


This automation showcases Elastic Beanstalk’s self-healing mechanism. The platform continuously monitors environment health through CloudWatch alarms and the underlying Auto Scaling Group. Whenever an instance fails or is manually terminated, the scaling policy triggers a recovery launch without human intervention. For developers, this eliminates the operational burden of managing uptime - proof of AWS’s managed-service resilience in action.

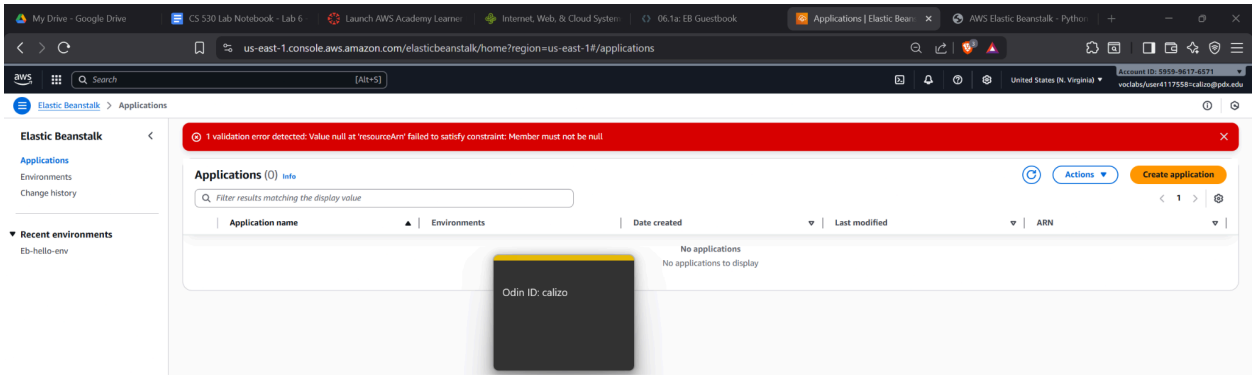
Step 5. Clean Up

After verifying the replacement process, I proceeded to decommission the Elastic Beanstalk environment to prevent unnecessary resource usage. From the Elastic Beanstalk console, I selected the eb-hello application, navigated to the environment dashboard, and chose Actions → Terminate environment.

Over the next few minutes, the console logs displayed sequential teardown events - deregistering instances from the load balancer, deleting the Auto Scaling Group, target group, security groups, and CloudWatch alarms. The status eventually updated to "Environment successfully terminated", confirming a complete and orderly shutdown.



Once the environment was terminated, I returned to the main Applications page and deleted the eb-hello application entirely. The console displayed a validation message and confirmed no applications remaining, signaling a fully cleaned workspace with no active environments or billing resources left behind.



Step 6. Elastic Beanstalk Guestbook

After deploying a sample app through the web console earlier, this step focused on performing a full Elastic Beanstalk deployment through the AWS CLI - mirroring what a CI/CD automation script would execute in production.

From Cloud Shell, I cloned the course repository and navigated into the 05_aws_dynamodb directory. Using the Elastic Beanstalk CLI, I initialized the environment with the Python 3.8 platform in the us-east-1 region through the command eb init -p python-3.8 guestbook --region us-east-1. This setup generated the configuration files necessary for EB to link to the current project.

Next, I executed:

- eb create guestbook-env --min-instances 3 --max-instances 10 --service-role LabRole --instance_profile LabInstanceProfile

This command provisioned all the resources required for the Guestbook service - three EC2 instances, load balancers, and associated IAM roles - within an Elastic Beanstalk-managed environment. The CLI logs confirmed successful initialization and reported the application URL once the environment stabilized.

The process was a live demonstration of automation through infrastructure-as-code principles: by defining operational parameters directly in the command, AWS dynamically constructed a scalable application backbone without any manual resource configuration.

Step 7. Deploying the Guestbook

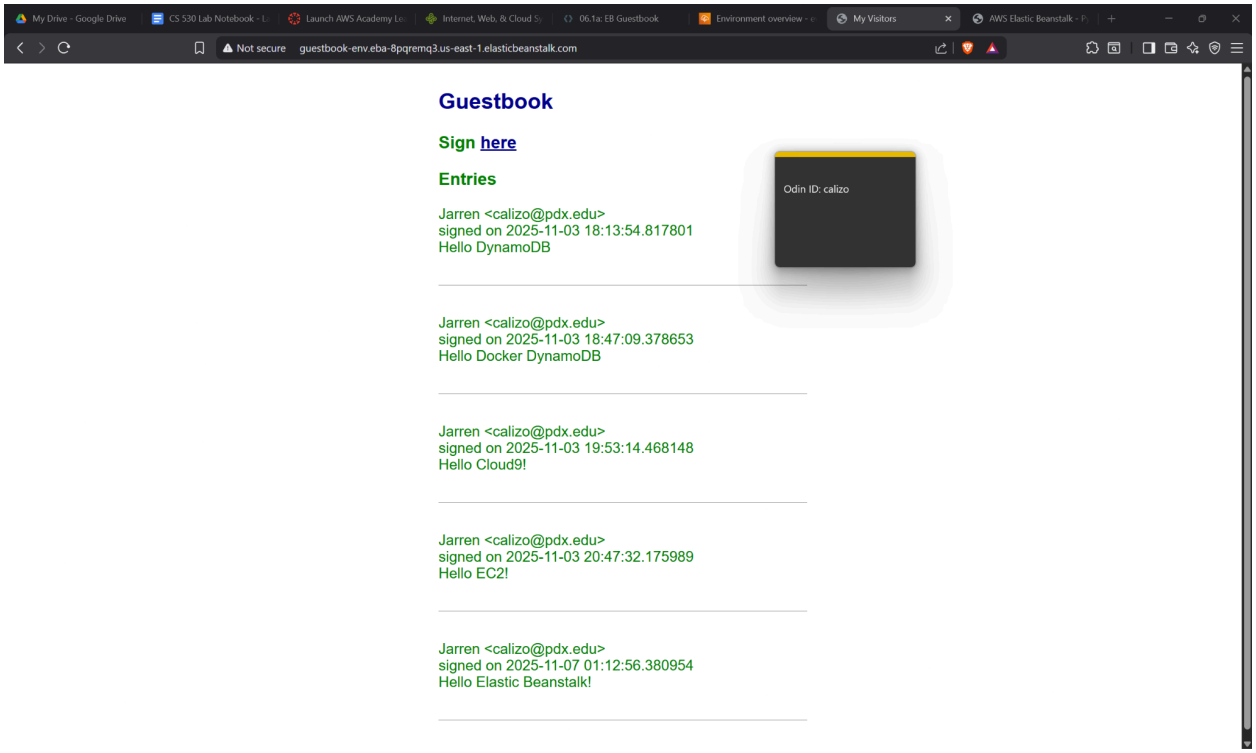
With the environment ready, I ran the deployment command:

- eb deploy guestbook-env

Elastic Beanstalk packaged the Guestbook source code, created a new application version, and rolled it out across all three running instances.

When the deployment completed, I visited the environment URL displayed in the CLI output. The Guestbook page loaded successfully, and I signed the guestbook using my name, PSU email address, and the message "Hello Elastic Beanstalk!".

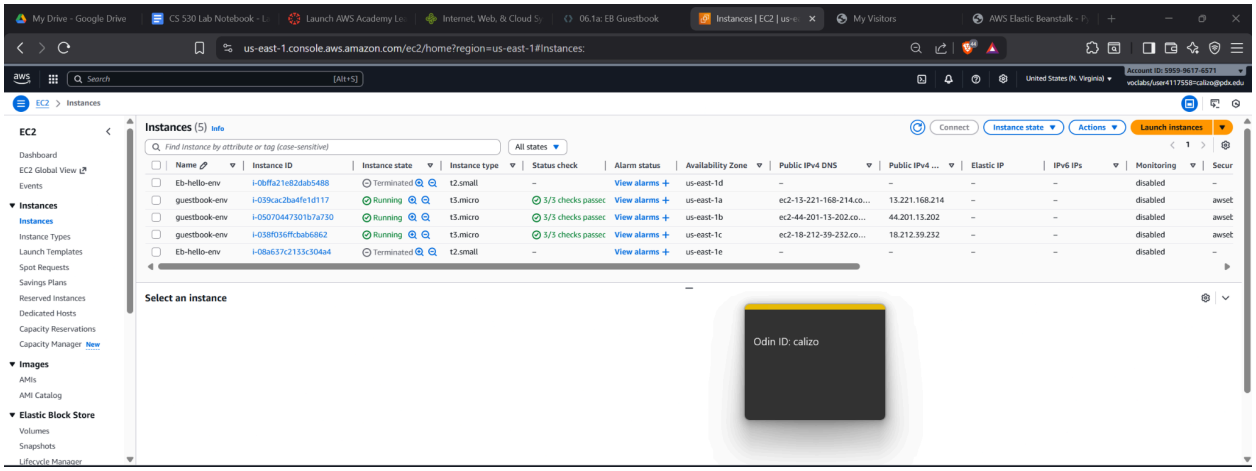
1. Take a screenshot of the Guestbook including the URL with the entry in it.



This is displaying the deployed Guestbook entries list and the environment URL confirming a successful deployment.

After verifying the web app functionality, I navigated to the EC2 console to inspect the running instances. Three new instances tagged under guestbook-env were running simultaneously, each distributed across different availability zones (us-east-1a, us-east-1b, and us-east-1c). The scaling configuration successfully honored the --min-instances 3 directive from the initialization command.

2. Take a screenshot of them.



This picture is validating the presence of three active instances created by Elastic Beanstalk for the Guestbook deployment.

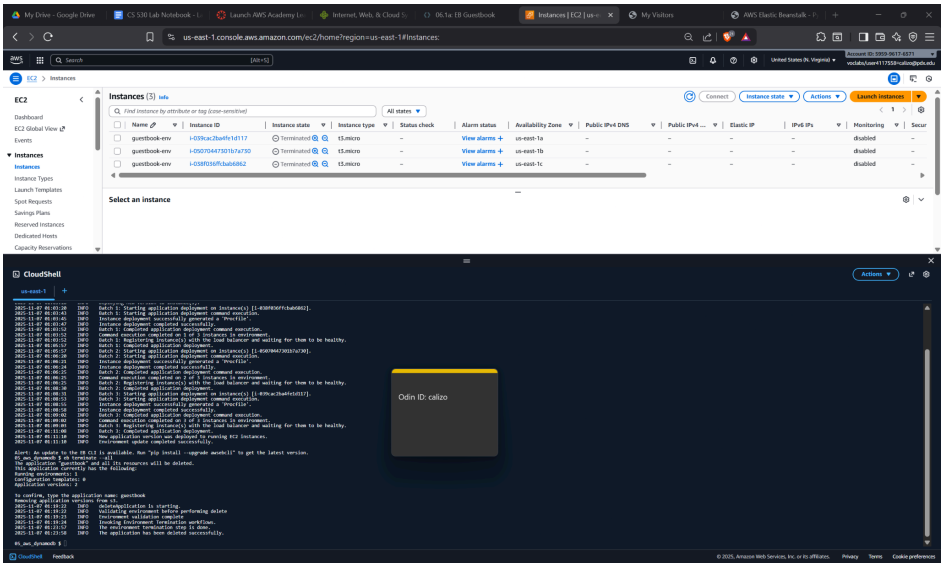
Step 8. Clean Up

To conclude the lab, I executed:

- eb terminate --all

This command gracefully tore down all Elastic Beanstalk resources, including the environment, load balancer, and EC2 instances. The Cloud Shell output confirmed that both the environment and the application were deleted successfully.

In the EC2 console, all Guestbook instances transitioned from running to terminated, indicating that the infrastructure had been fully decommissioned.



This final step reinforced the importance of responsible cloud resource management. Even though Elastic Beanstalk automates deployment and scaling, it also expects engineers to explicitly handle cleanup - preventing unnecessary resource costs and maintaining a clean AWS workspace.

Insight

Deploying the Guestbook through the Elastic Beanstalk CLI encapsulated the essence of modern DevOps workflows. The process transitioned smoothly from code to cloud, showcasing the scalability and automation capabilities of AWS. Observing the environment scale itself across multiple zones and then dismantle with a single termination command was an elegant reminder of how infrastructure today is both ephemeral and programmable - provisioned in minutes, retired in moments, and entirely scriptable from a single shell.

Lab 6.1g App Engine Guestbook

Step 1. App Engine

Google App Engine is a Platform-as-a-Service (PaaS) environment designed to abstract away infrastructure, letting developers focus purely on application logic. Similar to AWS Elastic Beanstalk, App Engine automatically handles provisioning, scaling, and fault tolerance - yet operates entirely within Google's ecosystem of managed services.

To begin, I enabled the App Engine API through Cloud Shell using the command gcloud services enable appengine.googleapis.com. Once activated, the App Engine environment became accessible under my project in the console. This laid the groundwork for deploying the Guestbook application in a serverless and scalable environment.

Step 2. app.yaml

Next, I navigated to the project’s datastore application directory:

- cd cs430-src/05_gcp_datastore

Here, I created an app.yaml file specifying a Python 3.11 standard environment and defined gunicorn as the entrypoint. The configuration also included manual scaling to two instances (manual_scaling: instances: 2) and allocated resources suitable for small workloads (1 CPU, 0.5 GB RAM, and 10 GB disk).

This YAML file acted as a blueprint for App Engine’s automated build process, allowing Google Cloud to containerize the Guestbook app and deploy it seamlessly without requiring a Dockerfile. Once finalized, the configuration was verified before deployment.

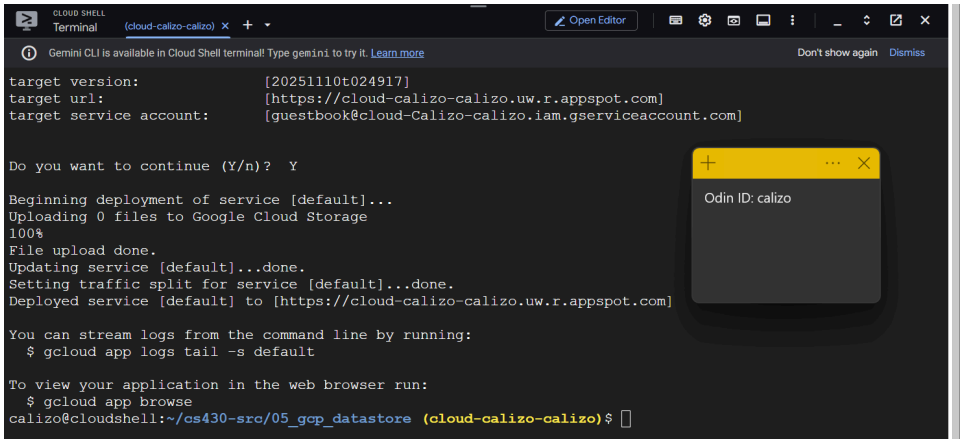
Step 3. Deploying the Guestbook

To deploy the app, I ran:

gcloud app deploy
--service-account=guestbook@cloud-Calizo-calizo.iam.gserviceaccount.com

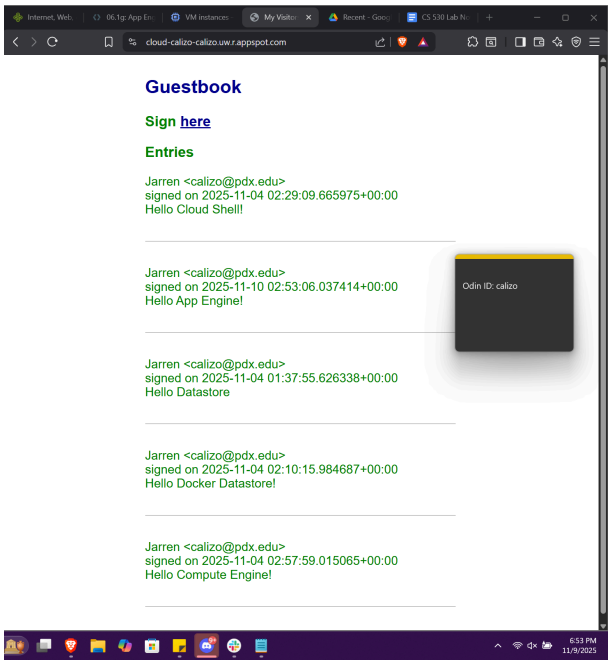
App Engine automatically uploaded the files, built the runtime container, and provisioned the necessary infrastructure - instances, load balancers, and networking. The deployment logs confirmed the app was live at:

- https://cloud-calizo-calizo.uw.r.appspot.com



Once the deployment completed, I opened the URL in a browser. The Guestbook page loaded successfully, and I signed it with my name and PSU email along with the message "Hello App Engine!". The new entry appeared instantly, verifying a live and functional connection to Cloud Datastore.

1. Take a screenshot of the output that includes the URL in the address bar for your lab notebook.

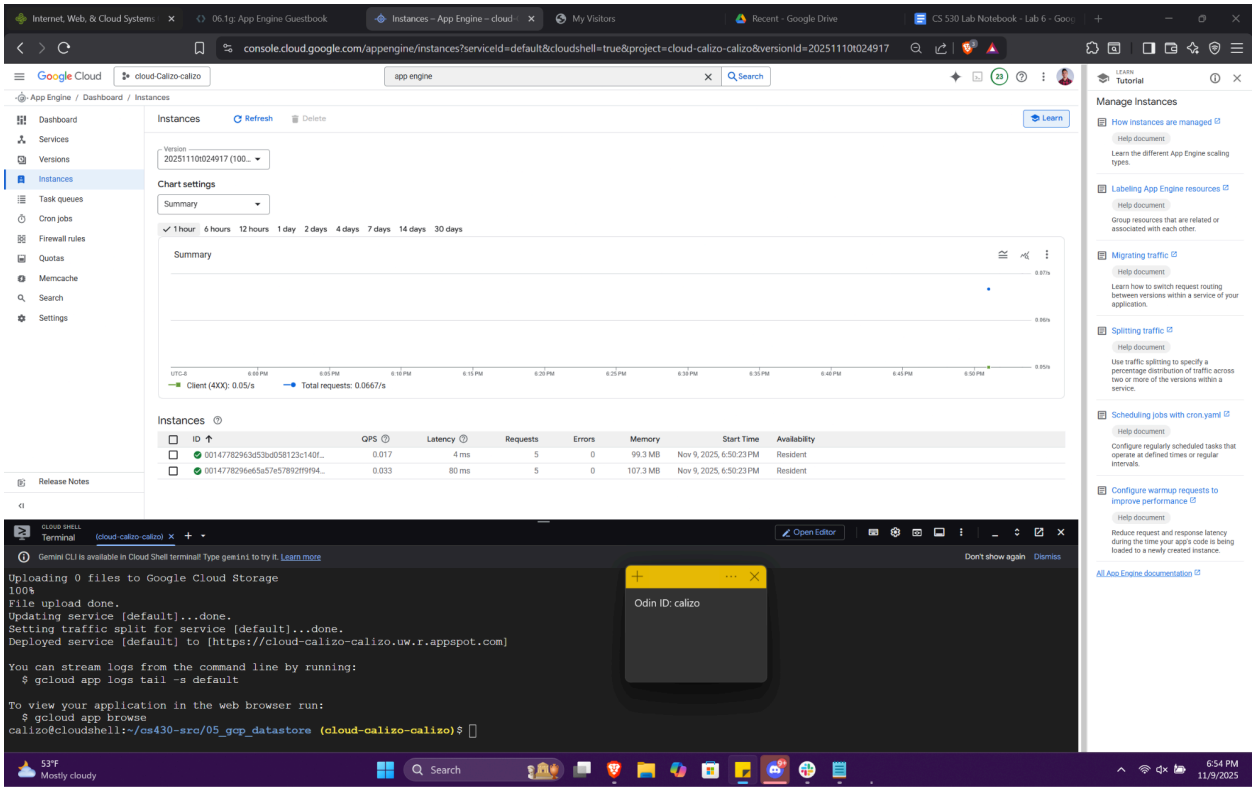


This is confirming a successful App Engine deployment and visible Guestbook entry.

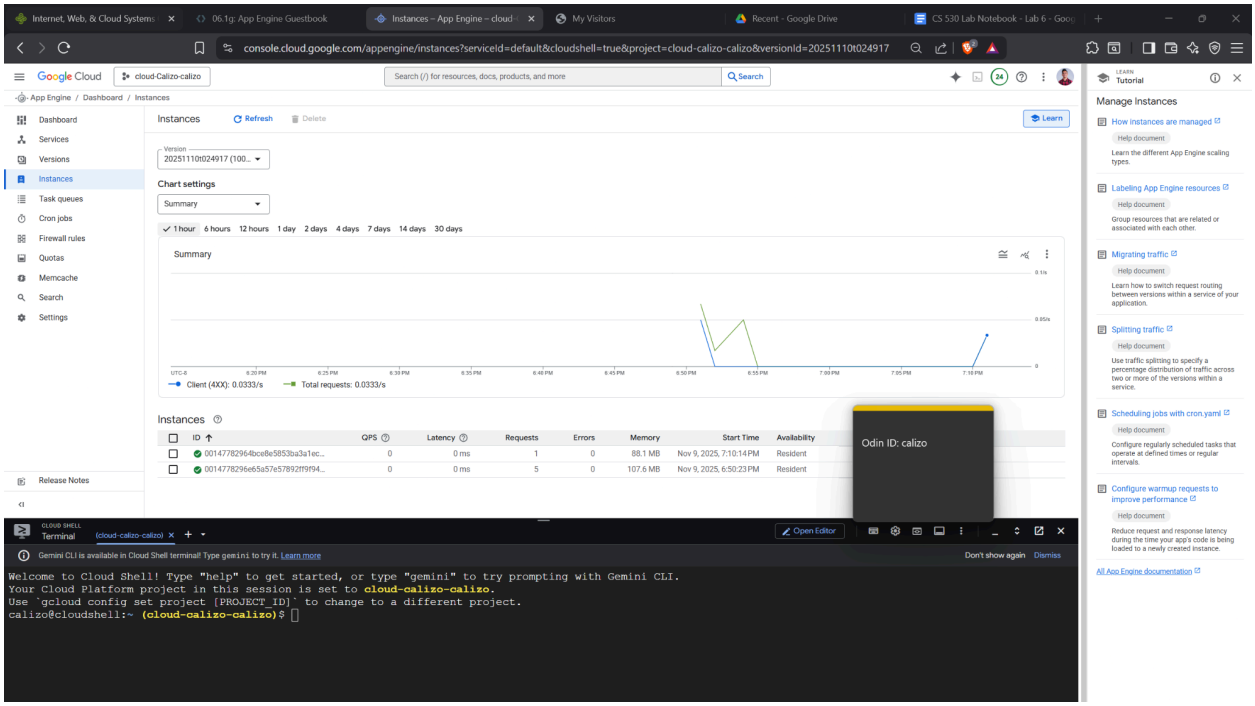
Step 4. Handling failures seamlessly

To observe fault tolerance, I visited App Engine → Instances in the GCP console. The dashboard showed two running replicas under the current version, consistent with the scaling configuration in app.yaml. Each instance reported independent performance metrics such as QPS, latency, and memory usage.

1. Take a screenshot of them.



When I deleted one instance, App Engine automatically launched a new one within moments to maintain the configured replica count. This seamless replacement demonstrated how App Engine continuously ensures availability without any manual infrastructure management.



These screenshots are showing both instances and the auto-replacement process triggered by App Engine.

Step 5. Clean Up

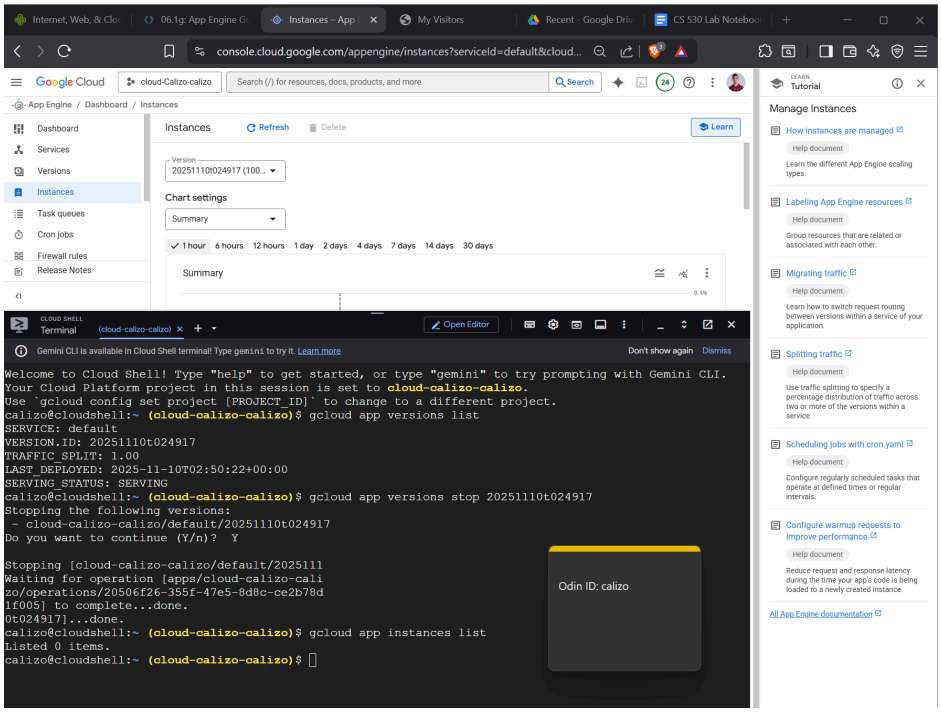
To responsibly shut down resources, I first listed the active versions:

- gcloud app versions list

The command displayed the deployed version ID (20251110t024917). I then stopped it with:

- gcloud app versions stop 20251110t024917

The output confirmed that the environment was stopped successfully. Verifying from the console under App Engine → Instances, no active instances remained.



This final step ensured that no residual App Engine resources were consuming compute or billing hours.

Insight

Deploying the Guestbook on App Engine illustrated the elegance of full serverless orchestration. From a single YAML configuration and CLI command, Google Cloud provisioned a highly available, auto-scaling, and monitored web service with built-in recovery logic. Watching the instance self-heal after deletion underscored the reliability of managed cloud systems - offering both simplicity and resilience at scale. In essence, this lab encapsulated the philosophy of modern cloud computing: automation, minimal friction, and complete focus on application delivery.

Lab 6.2g Cloud Run, Secret Manager (Web proxy)

Step 1. Cloud Run

Cloud Run represents one of Google Cloud's most versatile serverless offerings, blending the flexibility of containers with the simplicity of managed infrastructure. Unlike App Engine, which restricts supported environments, Cloud Run allows developers to deploy any application that can be containerized - granting greater control over runtime configurations.
To start, I enabled the Cloud Run service using gcloud services enable run.googleapis.com, confirming that the API activated successfully. This established the foundation for deploying a fully containerized application with autoscaling and stateless execution.

Step 2. Application Setup

I structured the application directories within Cloud Shell to align with the provided hierarchy:

```
secret-proxy/
├── app.py
├── Dockerfile
├── requirements.txt
└── templates/
    └── proxy.html
```

This setup cleanly separated the main Python application, its dependencies, and the HTML front-end template. Using the commands mkdir -p secret-proxy/templates and cd secret-proxy, I created and entered the working directory, preparing for subsequent file definitions.

Step 3. app.py

The main application file implemented two core routes using Flask:

1. / - a simple "Hello World" landing page.
2. A secret proxy route defined dynamically via the environment variable SECRET_PROXY_ROUTE.

When the environment variable was set, Flask registered a hidden route that handled requests by checking for a url parameter. If provided, it fetched and returned the target webpage using the requests library; otherwise, it displayed the proxy.html input page.

This conditional route creation reinforced an important security concept - keeping sensitive paths hidden until explicitly injected through runtime configuration.

Step 4. templates/proxy.html

Next, I implemented a simple interactive proxy interface under templates/proxy.html. The HTML and embedded JavaScript captured user input, listening for the Enter key to redirect the browser to the proxy route appended with a ?url= parameter. This allowed the user to seamlessly fetch remote content through the proxy once the secret route was known.

Step 5. requirements.txt

I then created a minimal dependency list for Flask, Requests, and Gunicorn:
flask
requests
gunicorn

These libraries collectively enabled the web application to handle routing (Flask), perform remote content retrieval (Requests), and serve production-grade traffic via Gunicorn.

Step 6. Dockerfile

The Dockerfile defined the environment for our containerized application.
Using python:3.10-slim as the base image kept the footprint lightweight. The image copied all files into /app, set the working directory, installed dependencies via pip install -r requirements.txt, and launched the Flask app using Gunicorn with the following command:

- CMD gunicorn --bind :$PORT --workers 1 --threads 8 app:app

This approach adhered to container best practices, using environment variables (like $PORT) for flexible deployment and single-threaded efficiency appropriate for testing.

Step 7. Build and Local Test in Cloud Shell

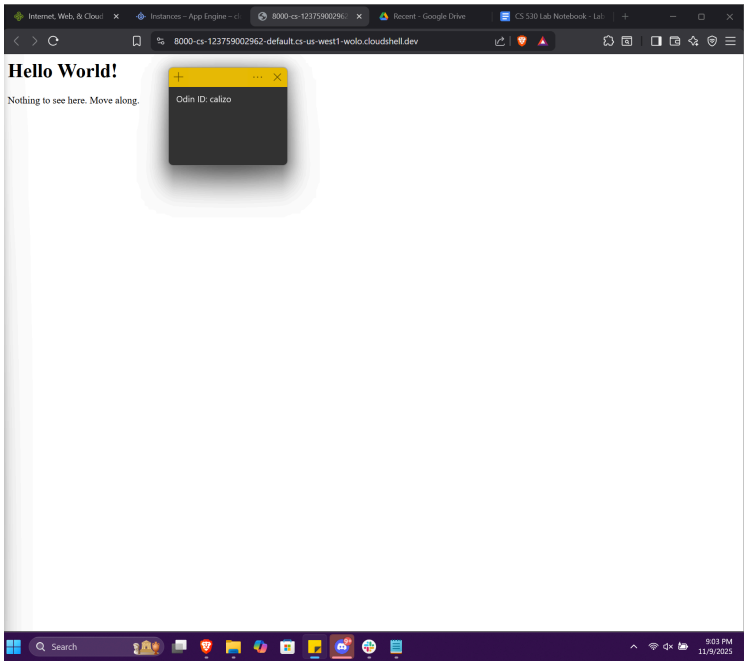
After building the image locally with:

- docker build -t secret-proxy-image ./

I confirmed a successful build and launched the container interactively:

- docker run --env PORT=8000 -p 8000:8000 --name=proxy --rm -it secret-proxy-image

Opening Web Preview on port 8000 revealed the expected landing page message: “Hello World! Nothing to see here. Move along.”



This validated that both Flask routing and Gunicorn served responses correctly within the containerized environment. After verification, I terminated the container with Ctrl + C, and the --rm flag automatically cleaned up the instance.

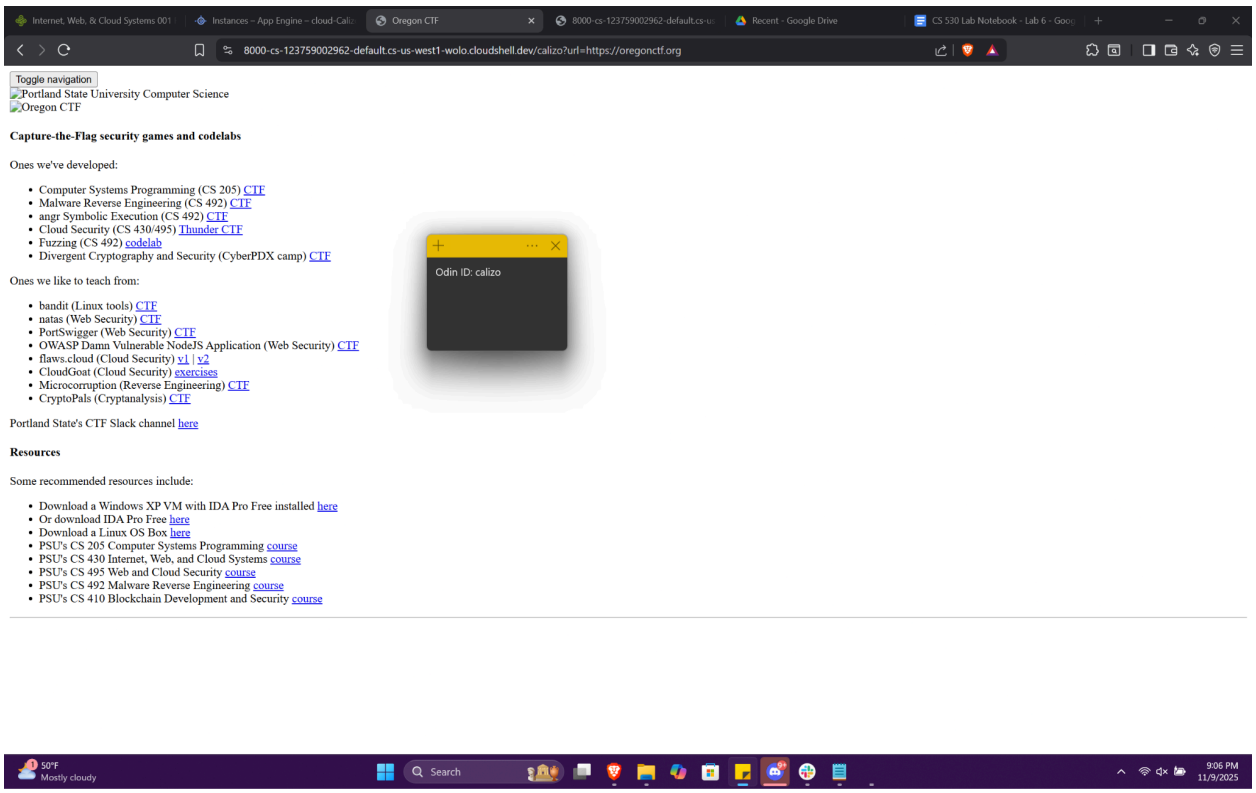
Step 8. Setup secret proxy

To activate the hidden route, I relaunched the container with the SECRET_PROXY_ROUTE environment variable set:

- docker run --env PORT=8000 --env SECRET_PROXY_ROUTE=/calizo -p 8000:8000 --name=proxy --rm -it secret-proxy-image

Visiting the new route /calizo in the browser opened the proxy input interface. Entering the URL https://oregonctf.org and pressing Enter displayed the fetched Oregon CTF webpage content, successfully verifying end-to-end proxy behavior.

1. Take a screenshot of the proxy and its results including the URL containing your OdinID



This shows the proxied Oregon CTF content accessed through my secret route /calizo, confirming the functionality of the environment-variable-based path.

2. What is the security advantage of passing in the secret proxy route as an environment variable?

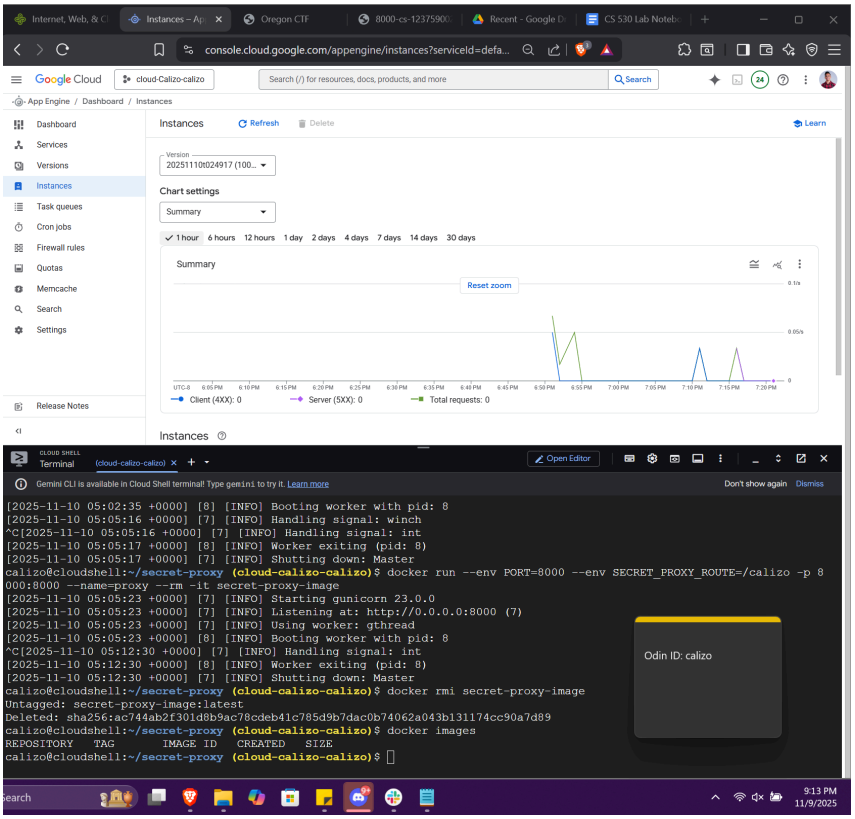
By passing the proxy route as an environment variable rather than hardcoding it, the route remains concealed from source code, repositories, and the container image itself. Only runtime environments with explicit configuration know the route’s value.

This minimizes the exposure of sensitive endpoints and protects against unauthorized discovery or source-level scanning attacks. The approach aligns with best practices in secure DevOps - keeping configurations external, secrets ephemeral, and codebases clean.

Once verified, I stopped the container and removed the image using:

- docker rmi secret-proxy-image

Listing images with docker images confirmed the repository had been deleted, ensuring no lingering artifacts remained in the Cloud Shell environment.



<u>Insight</u>

This lab illustrated the power of Cloud Run’s container-driven flexibility and the discipline of isolating configuration through environment variables. The simplicity of Docker coupled with Flask’s adaptability showcased how developers can build dynamic applications without compromising security. Watching the secret proxy come to life - from “Hello World” to live web relay - encapsulated the essence of controlled,

modern serverless architecture: rapid deployment, isolation by design, and security through configuration.

Step 9. Artifact Registry

To extend our container workflow beyond local Docker builds, I enabled Artifact Registry and Cloud Build to automate and securely host images within Google Cloud’s private repository infrastructure.

After activating the services via Cloud Shell, I created a regional Docker repository named secret-proxy-repo in us-west1. Using gcloud artifacts repositories describe, I verified the repository’s metadata and confirmed its readiness for integration with Cloud Build.

This marked the transition from a local build model to a DevOps-oriented cloud-native build process where artifacts are securely managed and isolated per project scope.

Step 10. Cloud Build

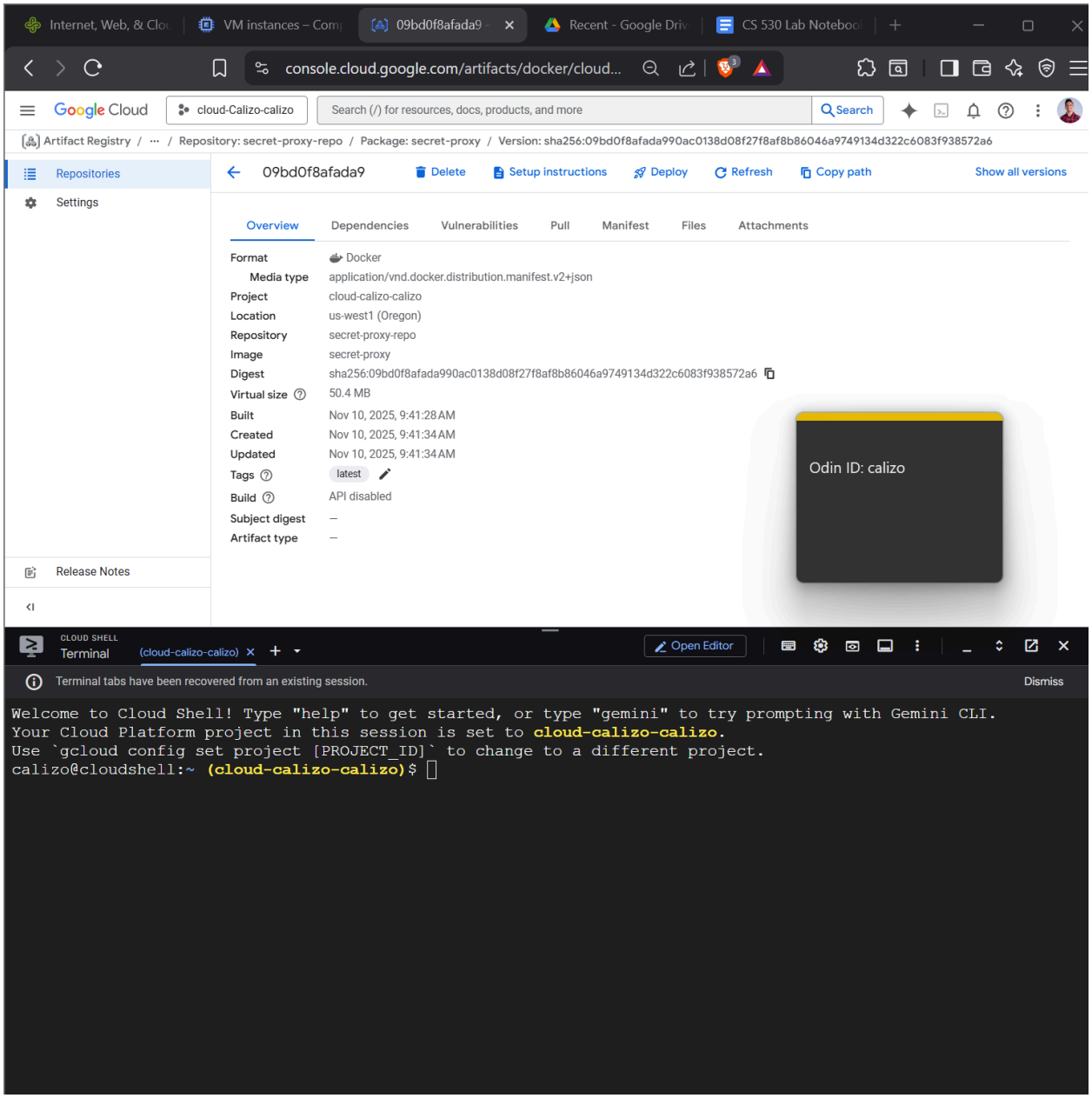
The next step was to automate the image build and store it within the new registry. Running:

- gcloud builds submit --tag us-west1-docker.pkg.dev/cloud-calizo-calizo/secret-proxy-repo/secret-proxy

triggered Cloud Build to fetch dependencies, layer the Dockerfile, and push the resulting image into Artifact Registry.

Listing the registry contents confirmed the image upload and tagging, visible through the web console with build metadata including the digest and creation timestamps.

1. Take a screenshot of the image in the registry that shows the size of the container for your lab notebook.



This is showing the finalized container artifact and metadata from the build. At this point, the application is production-ready, stored securely in GCP's internal image registry, awaiting deployment through Cloud Run.

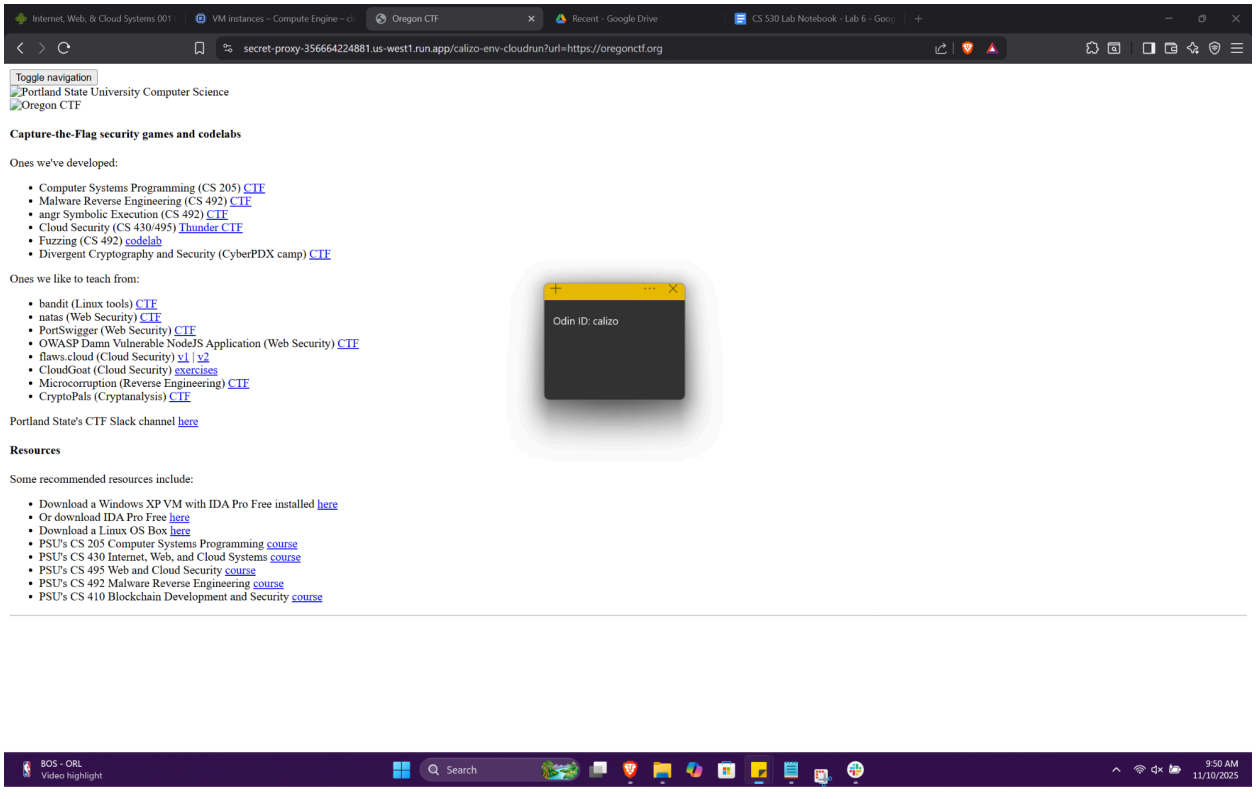
Step 11. Deploy to Cloud Run

Deployment began with the Cloud Run command that launched the containerized Flask proxy image from Artifact Registry, this time assigning the environment variable SECRET_PROXY_ROUTE=/calizo-env-cloudrun.

```
gcloud run deploy secret-proxy \
  --image
us-west1-docker.pkg.dev/cloud-calizo-calizo/secret-proxy-repo/secret-proxy \
  --platform=managed --region=us-west1 --allow-unauthenticated \
  --update-env-vars=SECRET_PROXY_ROUTE=/calizo-env-cloudrun
```

Once deployed, I received a new service URL under *.run.app, confirming successful provisioning. Accessing the route /calizo-env-cloudrun revealed the working proxy app and verified its ability to relay the Oregon CTF site content through the container.

1. Take a screenshot of it that includes the proxy URL for your lab notebook.



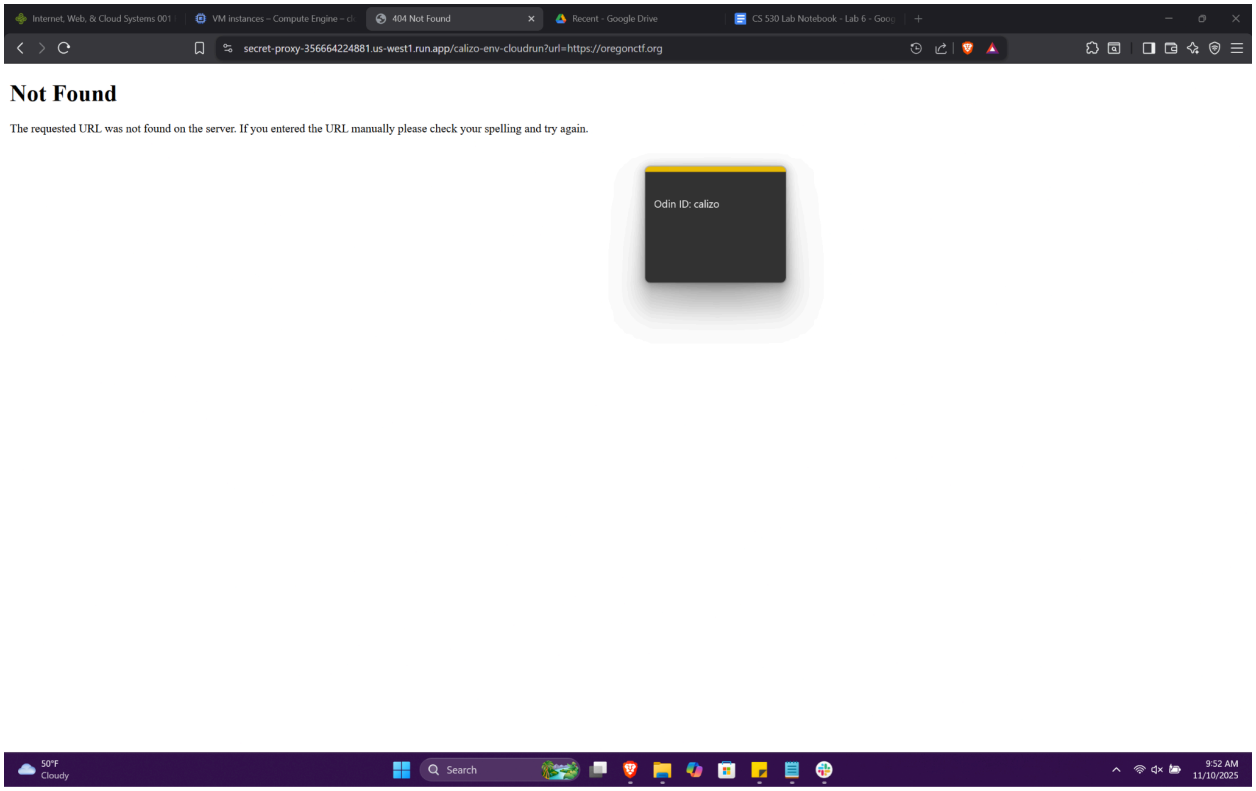
Following verification, I then removed the environment variable to simulate a route deactivation:

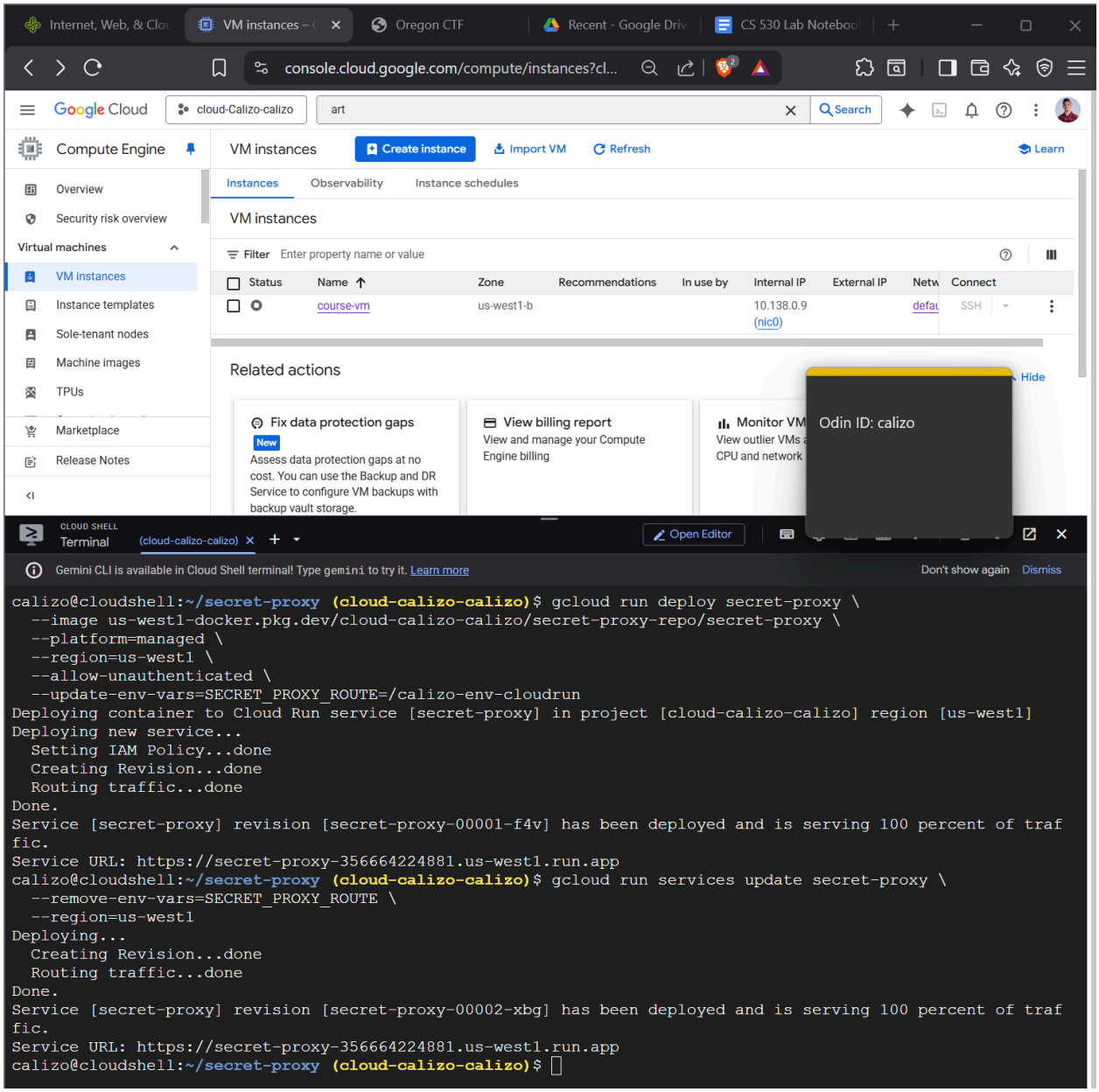
```
gcloud run services update secret-proxy \
```

```
  --remove-env-vars=SECRET_PROXY_ROUTE \
  --region=us-west1
```

Reloading the same route produced a “Not Found” message, confirming Cloud Run’s live configuration update in action.

2. Take a screenshot of the error page that includes the proxy URL for your lab notebook.





Step 12. Secret Manager

With basic environment-based deployment verified, I transitioned to a more secure configuration using Secret Manager.

I began by enabling the API and creating a secret called proxy-secret, storing the value /calizo-env-secretmanager:

- echo -n "/calizo-env-secretmanager" | gcloud secrets create proxy-secret --data-file=-

Next, I retrieved the Compute Engine default service account and granted it the Secret Manager Secret Accessor role. This step allowed Cloud Run to safely pull the secret value at runtime without embedding it directly into environment variables.

This configuration embodies secure secret rotation principles - separating sensitive route data from the codebase while still making it programmatically retrievable during deployment.

Step 13. Deploy to Cloud Run with Secret Manager

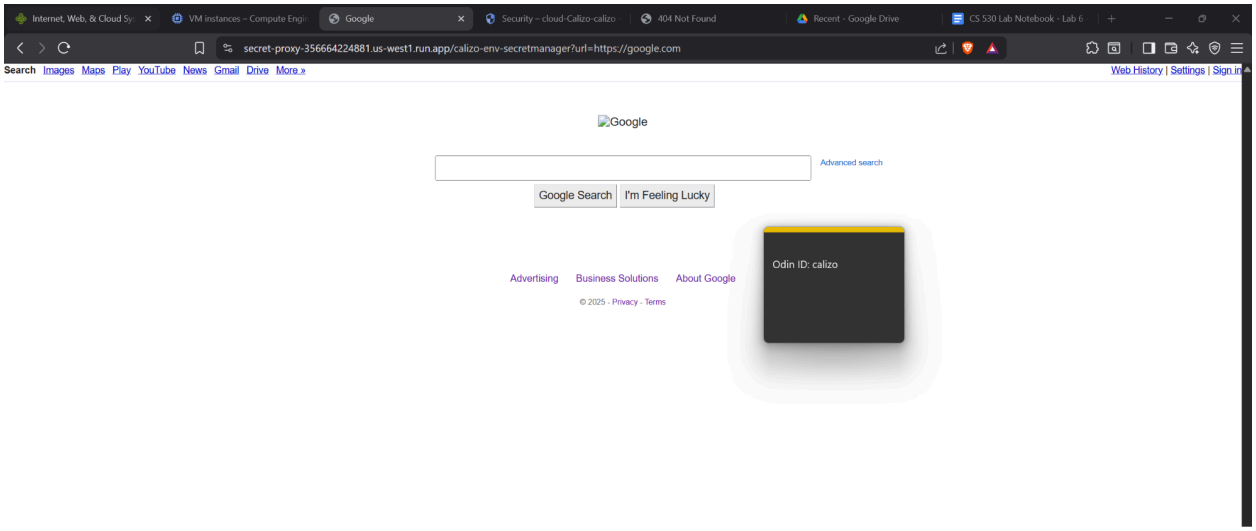
Step 13. Deploy to Cloud Run with Secret Manager

Using the newly created secret, I redeployed the container with the --update-secrets flag:

```
gcloud run services update secret-proxy \
  --update-secrets=SECRET_PROXY_ROUTE=proxy-secret:latest \
  --region=us-west1
```

After redeployment, I verified that the route /calizo-env-secretmanager was automatically injected through the secret reference. Visiting this new endpoint successfully rendered the proxied content once more, this time for https://google.com.

1. Take a screenshot of it that includes the proxy URL for your lab notebook.



The image above shows the application successfully fetching Google’s homepage through the Cloud Run URL configured via Secret Manager, confirming that the injected route /env-secretmanager worked correctly.

I then attempted to access the internal metadata endpoints:

- http://169.254.169.254/computeMetadata
- http://169.254.169.254/computeMetadata/v1

Both requests were blocked - a vital security measure.

2. Identify the vulnerability in your lab notebook that Google has prevented.

Google has mitigated Server-Side Request Forgery (SSRF) by restricting access to internal metadata endpoints (169.254.169.254). This prevents malicious requests from extracting sensitive instance information or credentials from the metadata service. By sandboxing containerized Cloud Run workloads, GCP enforces a strict isolation layer, ensuring SSRF vectors cannot compromise service accounts or IAM tokens.

Step 14. Clean Up

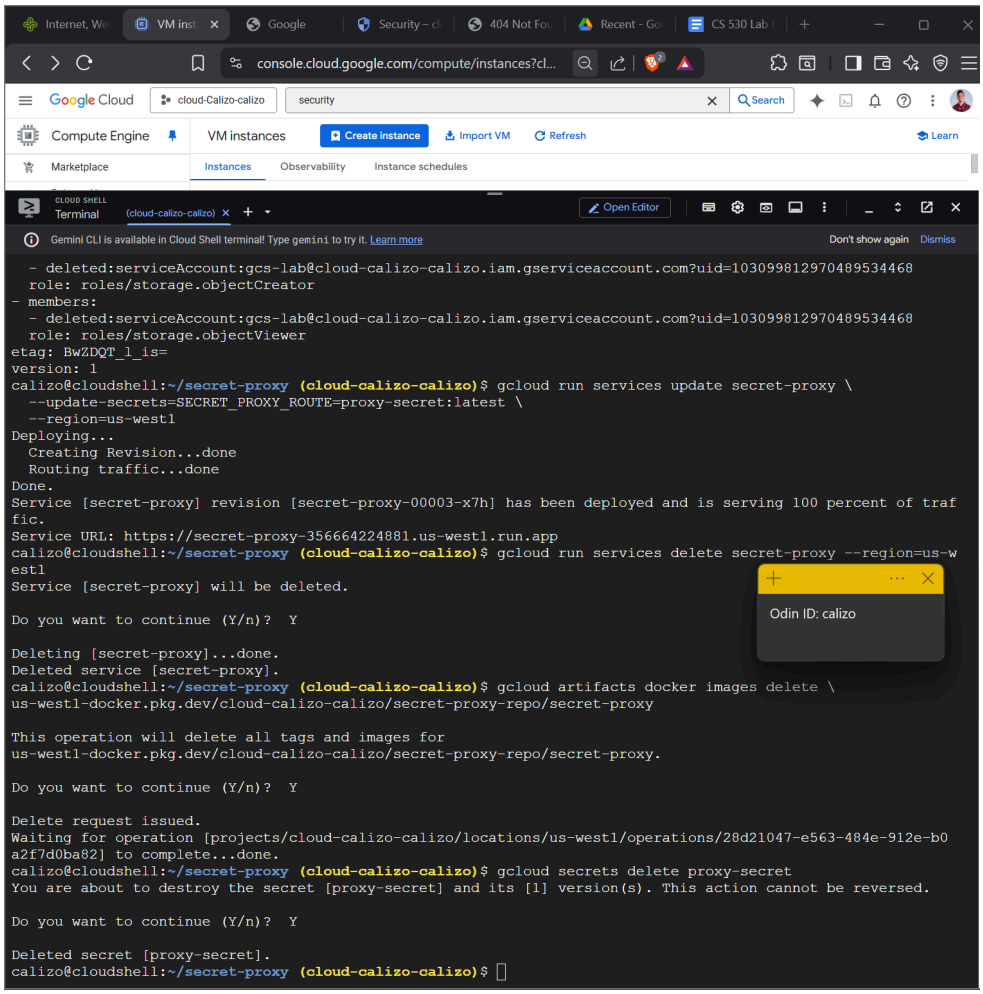
To conclude the lab, I performed a full teardown of all created resources through Cloud Shell commands:

- gcloud run services delete secret-proxy --region=us-west1

- gcloud artifacts docker images delete us-west1-docker.pkg.dev/cloud-calizo-calizo/secret-proxy-repo/secret-proxy
- gcloud secrets delete proxy-secret

Each deletion prompt confirmed the removal of Cloud Run services, Artifact Registry images, and stored secrets.

This step restored my project environment to its pre-lab state and avoided unnecessary resource usage.

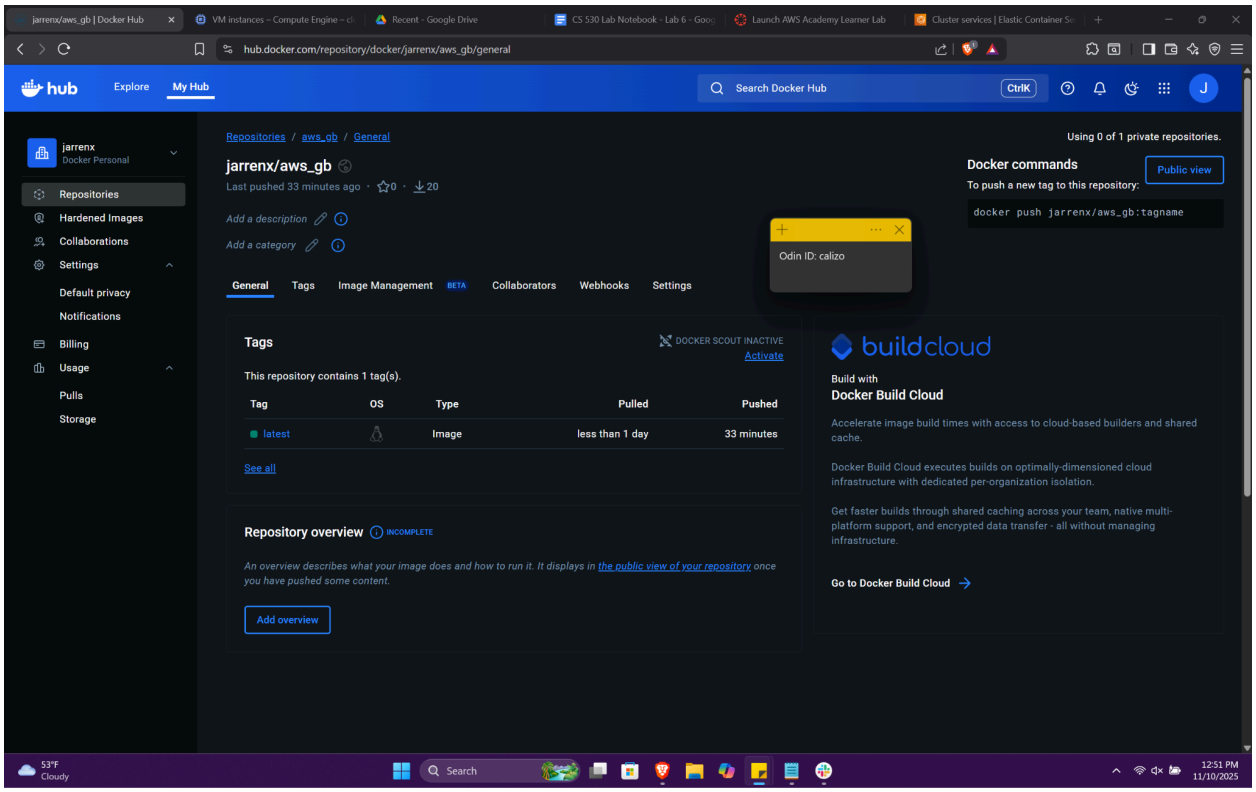


Lab 6.3a ECS Guestbook

Step 1. Prepare a container image

To begin, I cloned the course repository and navigated into the DynamoDB guestbook directory before reviewing the provided Dockerfile. After inserting my PSU email address as the maintainer, I built the container image using docker build -t aws_gb .. The resulting image was then tagged and pushed to my Docker Hub repository using my handle jarrenx/aws_gb.

Once pushed, I verified the image upload on Docker Hub, confirming that the container was successfully published under the latest tag. This ensures that ECS can later reference it through its public Image URI during task definition creation.



Step 2. ECS overview

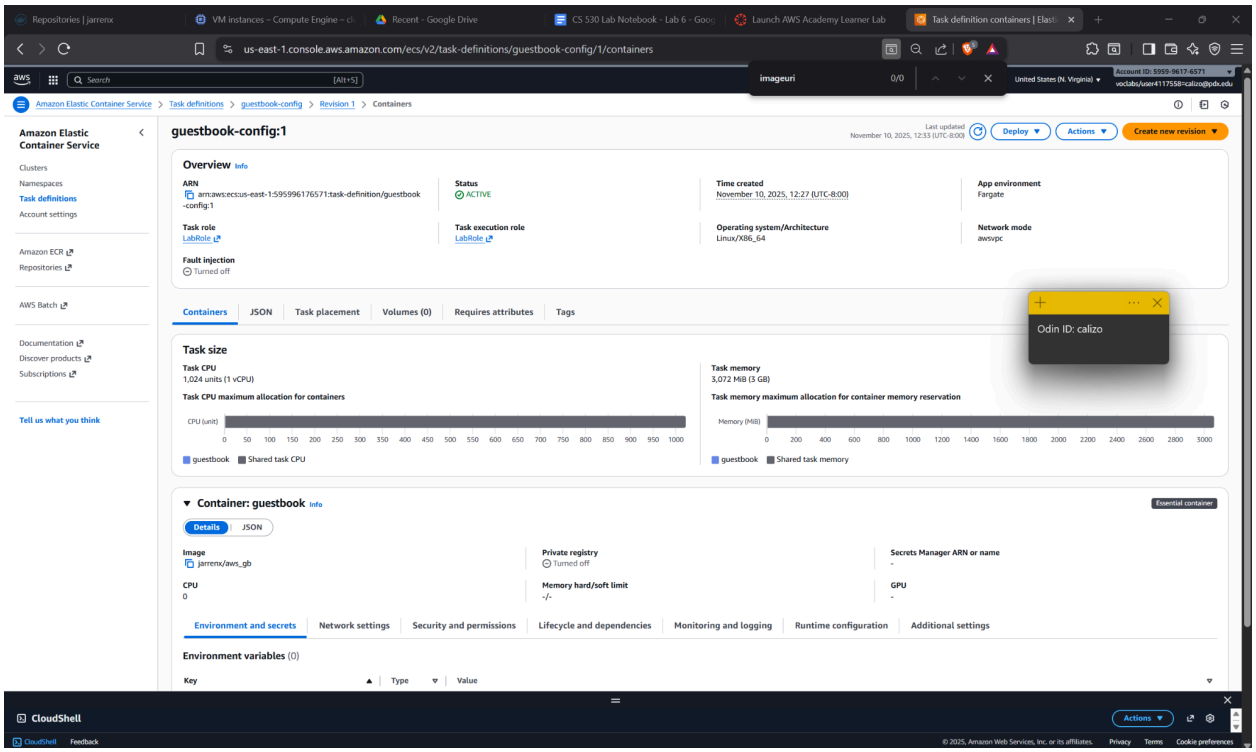
Amazon Elastic Container Service (ECS) provides a managed container orchestration platform. In this lab, I used AWS Fargate to run the guestbook application serverlessly - allowing the task to scale and execute without managing EC2 instances directly. The architecture centers around an ECS task definition (to specify container parameters),

an ECS service (to run and manage the task), and a load balancer (to expose it to the public internet).

Step 3. ECS task definition

In the ECS console, I created a new task definition named guestbook-config using the Fargate launch type. I attached the preconfigured LabRole, selected Linux/x86_64 as the operating system architecture, and set the container image to my Docker Hub URI jarrenx/aws_gb.

The configuration allocated 1 vCPU and 3 GB of memory. The container port mapping was set to 80/tcp to expose the web service to the load balancer. This configuration was successfully registered and marked as ACTIVE, confirming a functional base definition for ECS deployment.



Step 4. Create a Cluster and a Service

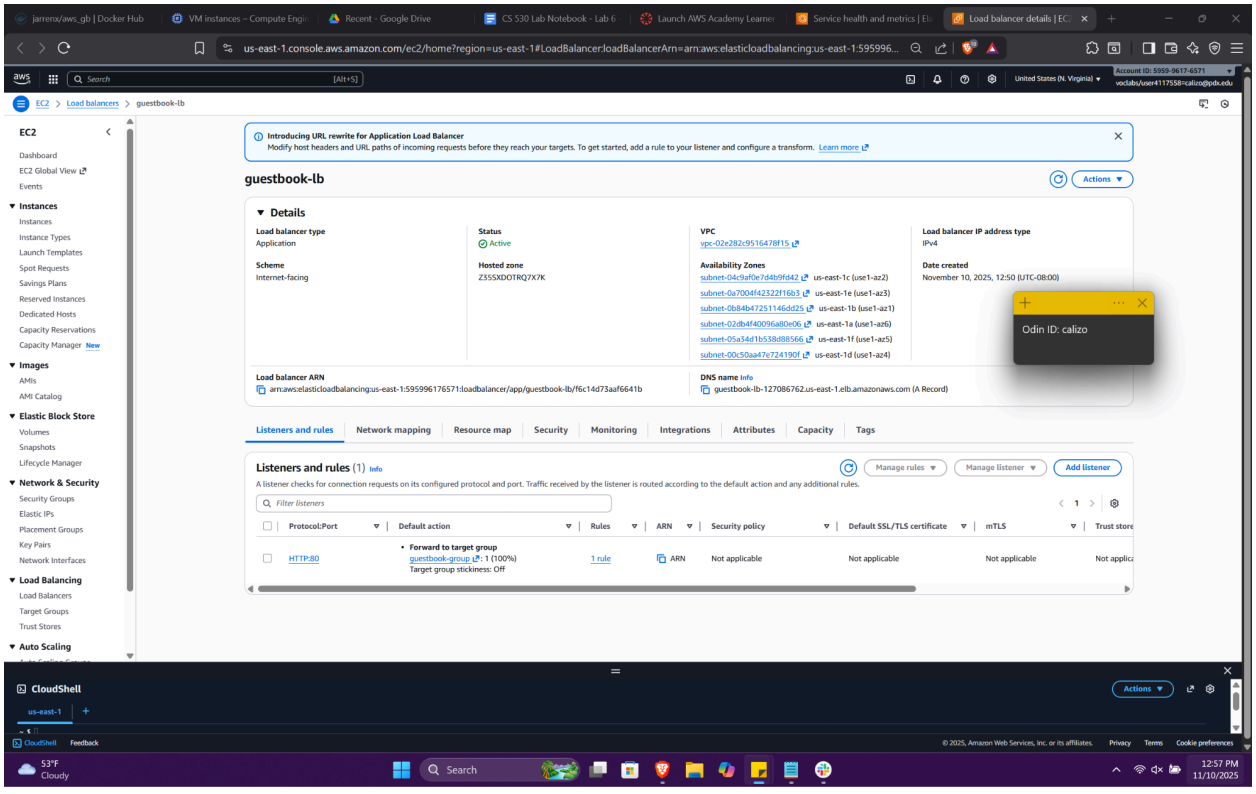
Next, I created a new ECS cluster named guestbook-cluster with the Fargate compute option and default VPC settings. Within this cluster, I defined a new ECS service titled guestbook-service, referencing the guestbook-config task definition.

I created a security group that allowed inbound HTTP traffic on port 80 from all IP addresses to make the guestbook application publicly accessible. Finally, I configured an Application Load Balancer (ALB) named guestbook-lb with a target group labeled guestbook-group, designed to route requests evenly to the running task container.

Step 5. Examine the service

1. Take a screenshot of the DNS name of the guestbook-lb load balancer for your lab notebook

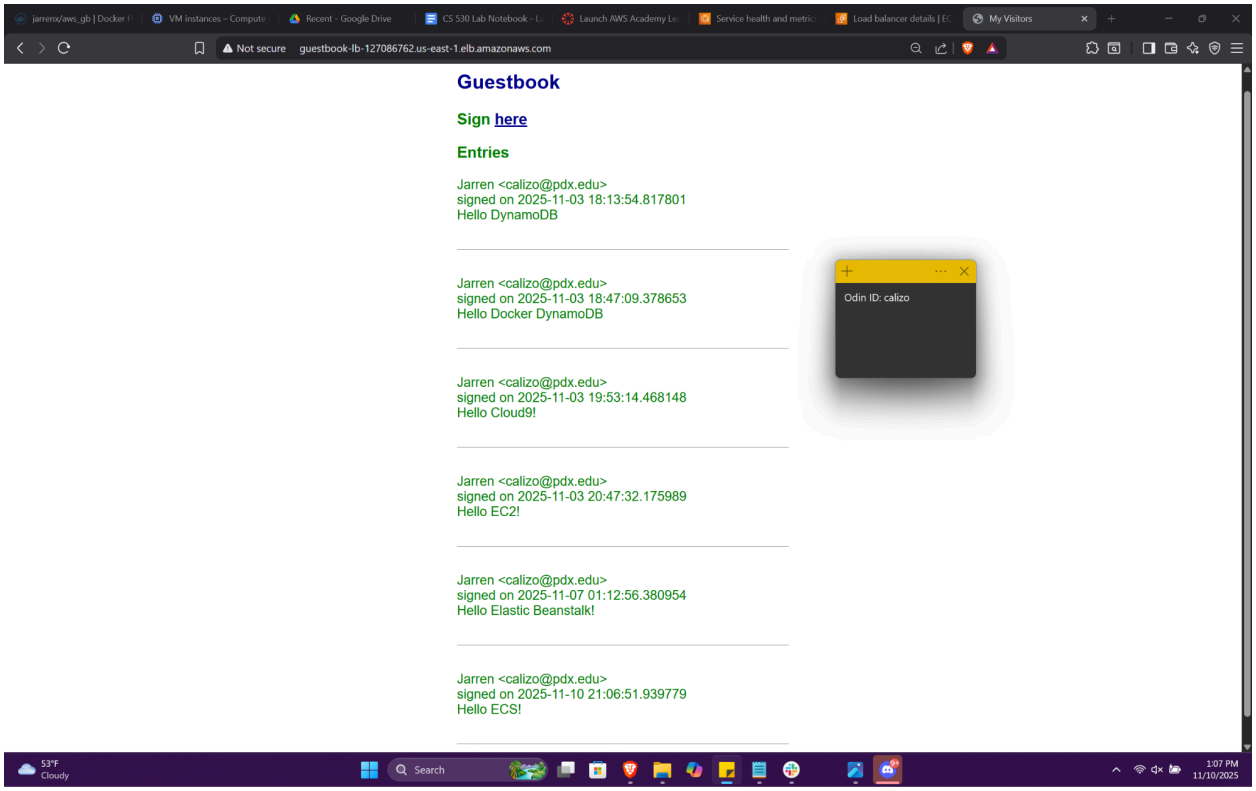
Once the ECS service launched successfully, I viewed the Load Balancer details under the EC2 dashboard. The ALB was shown as Active and internet-facing, with its DNS name serving as the external entry point for my application.



The DNS name of the load balancer is visible under its details section in the EC2 console and serves as the URL endpoint to access the guestbook web interface.

Step 6. Visit the site

1. Take a screenshot of the Guestbook app running in a browser that includes the DNS name of the site.



Using the DNS name of guestbook-lb, I opened the guestbook application in my browser. The web interface loaded successfully, showing prior DynamoDB entries from previous labs.

I then signed the guestbook with the new message "Hello ECS!", confirming that entries persisted dynamically through the existing DynamoDB backend connection. The live page with the new entry and DNS URL visible, proving that the ECS Fargate deployment and load balancing integration functioned correctly.

Step 7. Clean Up

To conclude, I decommissioned all resources to avoid incurring costs.

1. Deleted the guestbook-service using the Force delete option.
2. Removed the guestbook-cluster after the service termination.
3. Deregistered the guestbook-config task definition.

This completed the full lifecycle of deploying, testing, and cleaning up an ECS containerized guestbook application on AWS Fargate.

Insight

This lab bridged together containerization principles with real-world cloud orchestration using ECS and Fargate. Seeing the guestbook’s persistent entries transition seamlessly from Docker to AWS Fargate illustrated the power of infrastructure abstraction. Every layer - from the Docker Hub push to the load-balanced deployment - mirrored how DevOps pipelines manage applications in production at scale.

Lab 6.3g Cloud Run Guestbook

Step 1. Cloud Run

This lab introduced the concept of deploying a fully containerized Guestbook application - backed by Cloud Datastore - onto Google Cloud Run. Unlike previous labs that used Docker Hub for container hosting, this deployment leveraged Google Cloud's native services: Cloud Build for automated image creation and Artifact Registry for private container storage.

Step 2. Prepare a container image

From Cloud Shell, I navigated into the source directory for the Datastore-based Guestbook using:

- cd cs430-src/05_gcp_datastore

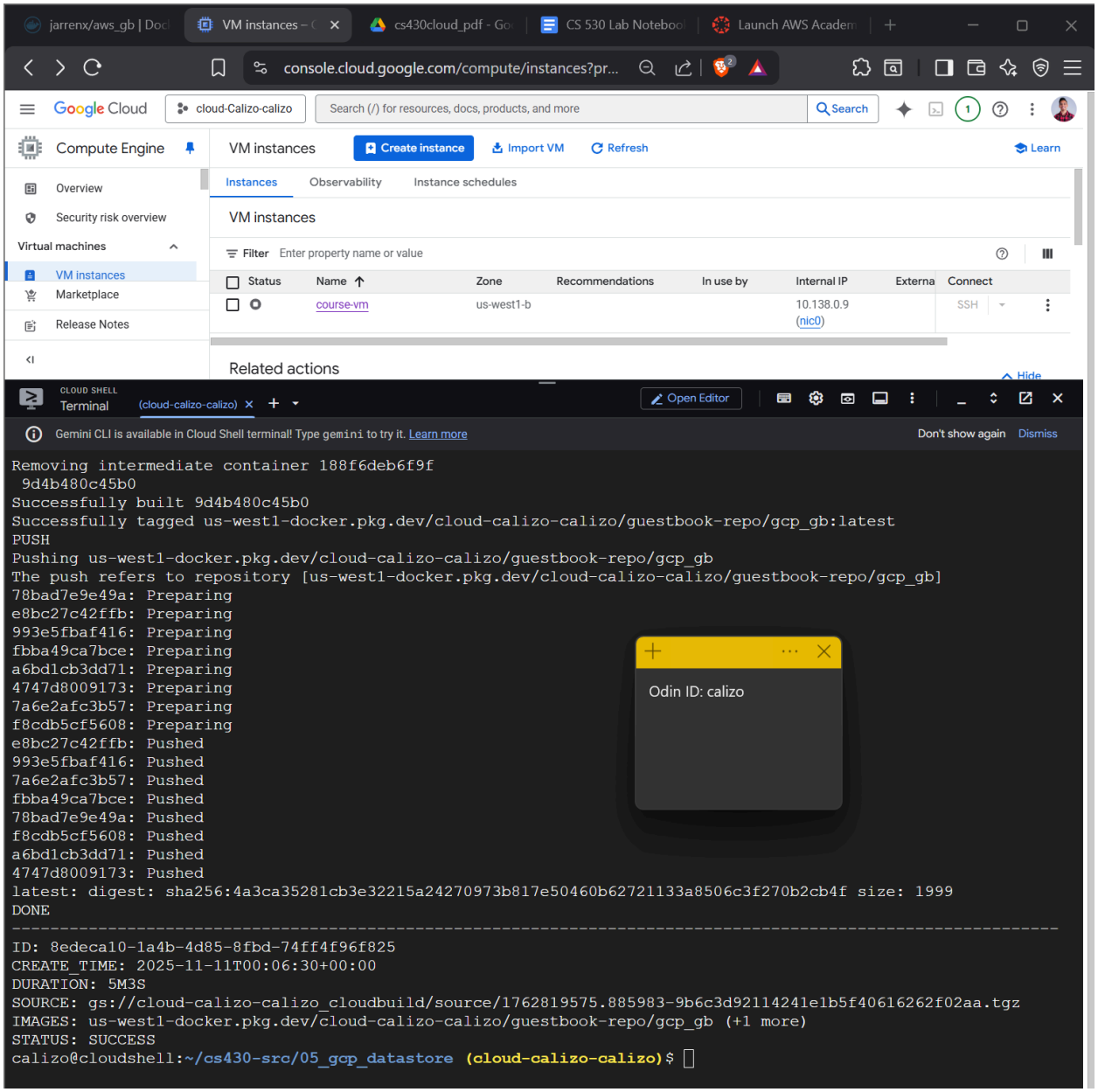
Then, I created a private container repository to store the image:

```
gcloud artifacts repositories create guestbook-repo \
--repository-format=docker \
--location=us-west1
```

Once confirmed, I executed the Cloud Build command to automatically construct and store the Docker image:

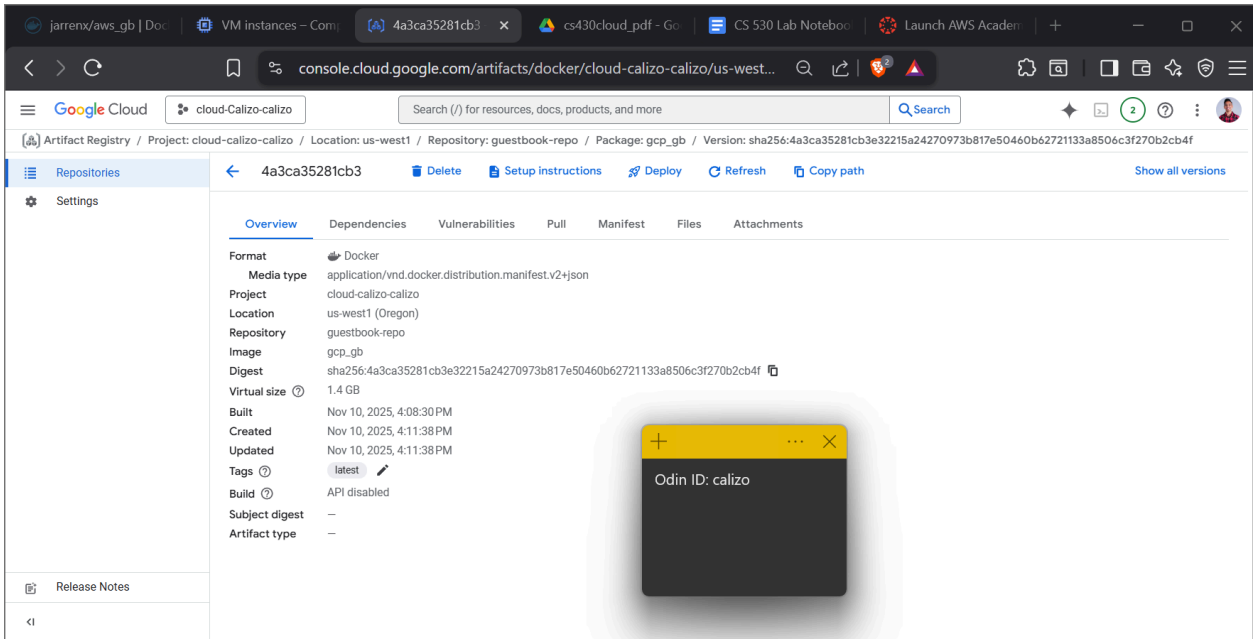
```
gcloud builds submit --tag \
us-west1-docker.pkg.dev/cloud-calizo-calizo/guestbook-repo/gcp_gb
```

1. Take a screenshot that includes the output of the command and the time it took to execute.



The terminal output confirmed successful image creation, showing the build duration and digest ID. The Cloud Build completed successfully within 5m3s, confirming the image upload to the Artifact Registry.

2. Take a screenshot showing the container image and its virtual size



After the build completed, I verified the resulting image in Artifact Registry. The stored image, titled gcp_gb, was listed with a virtual size of 1.4 GB and a Docker media type of application/vnd.docker.distribution.manifest.v2+json. In other words, the Artifact Registry confirmed the gcp_gb image with its metadata and storage information.

Step 3. Deploy container with minimal privileges

To enforce the principle of least privilege, the deployment utilized a preconfigured service account with only Cloud Datastore User permissions:

- guestbook@cloud-calizo-calizo.iam.gserviceaccount.com

From Cloud Shell, I deployed the container using Cloud Run:

```
gcloud run deploy gcp-gb \
--image us-west1-docker.pkg.dev/cloud-calizo-calizo/guestbook-repo/gcp_gb \
--service-account guestbook@cloud-calizo-calizo.iam.gserviceaccount.com
```

When prompted, I selected Cloud Run (fully managed), region us-west1, and enabled unauthenticated access. Cloud Run automatically set the appropriate internal PORT variable and launched the service.

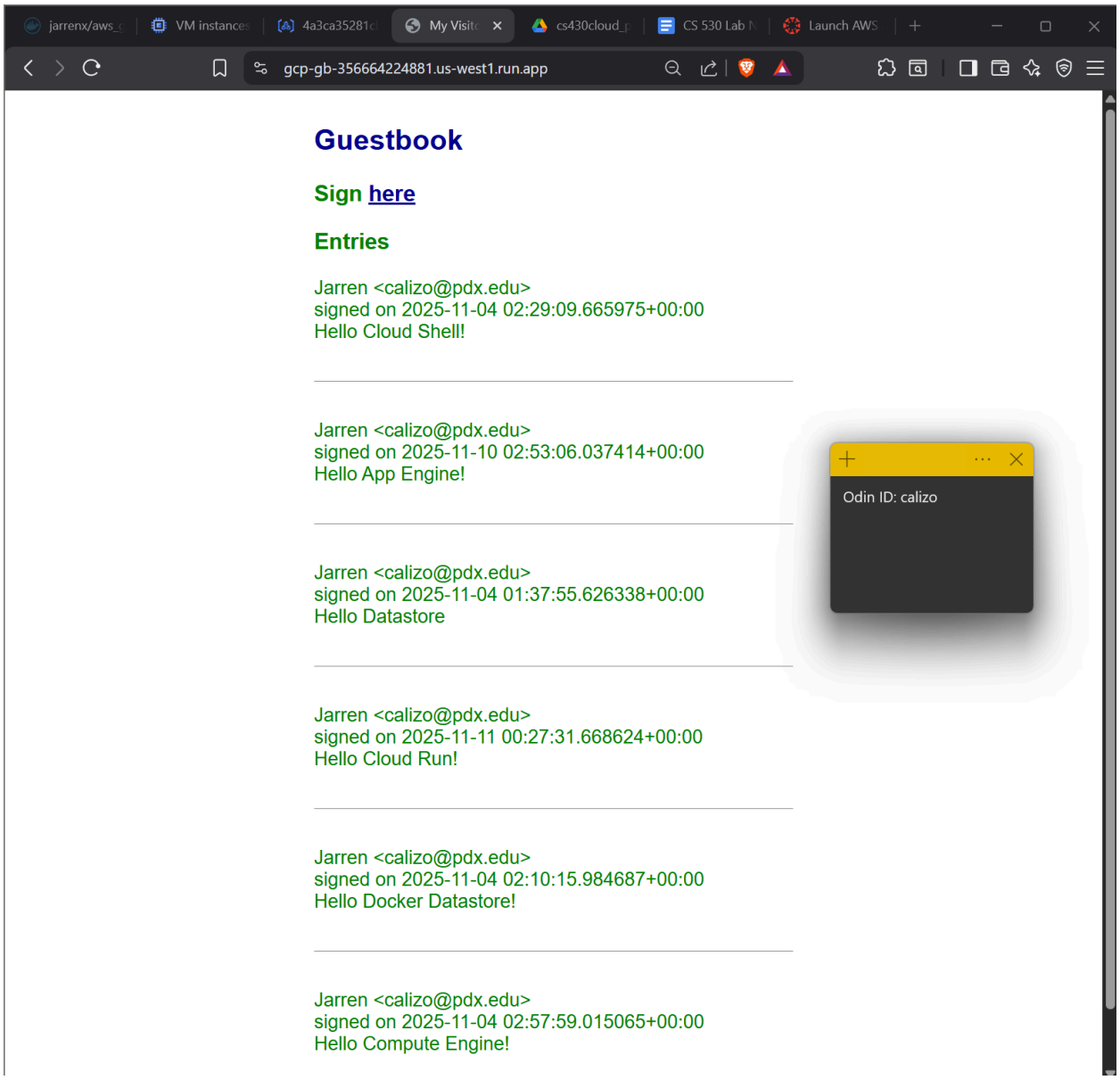
Step 4. View the Guestbook

After deployment, Cloud Run returned a public HTTPS endpoint:

- https://gcp-gb-356664224881.us-west1.run.app

Opening the URL in a browser revealed the familiar Guestbook interface populated with previous entries. I added a new message reading "Hello Cloud Run!", confirming full application functionality.

1. Take a screenshot that includes the URL Cloud Run has created for your site.



2. What port do container instances listen on?

The container instances listen on port 8080. Cloud Run automatically injects this PORT environment variable, allowing the application to bind correctly without manual configuration.

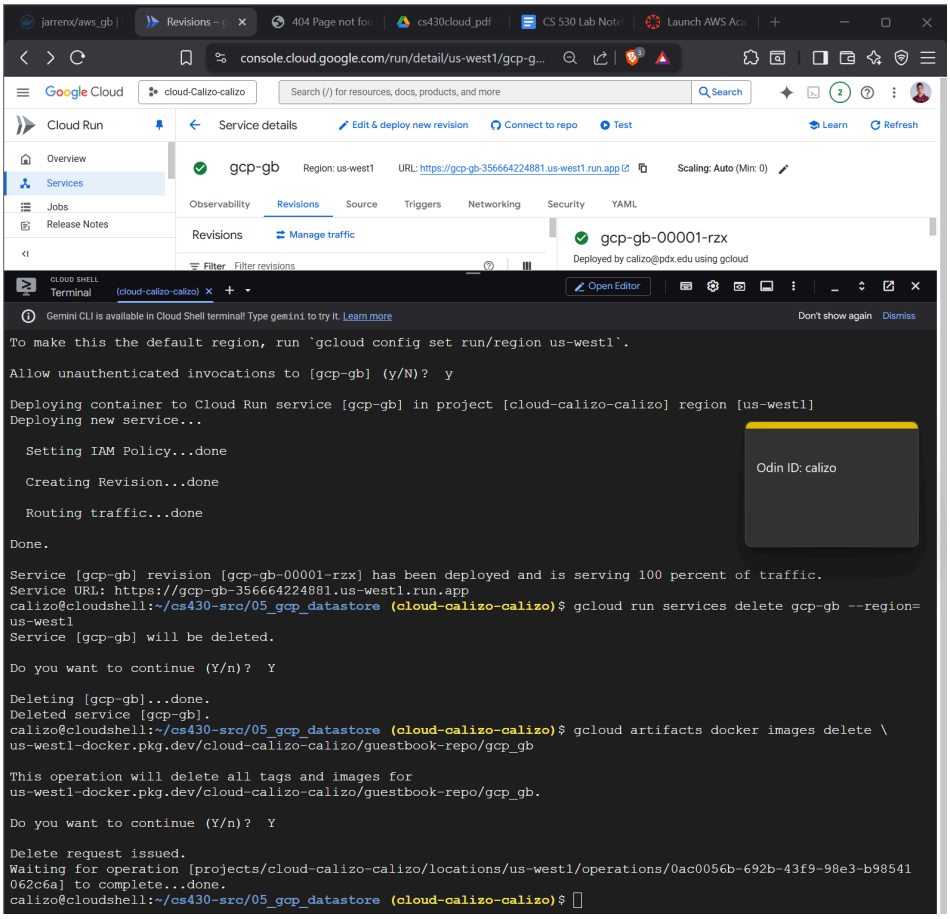
3. What are the maximum number of instances Cloud Run will autoscale up to for your service?

By default, Cloud Run can autoscale up to 100 instances, dynamically adjusting capacity based on incoming traffic and concurrency limits.

Step 5. Clean Up

After validating the deployment, I cleaned up all Cloud Run and Artifact Registry resources to avoid unnecessary costs. Using Cloud Shell, I executed:

gcloud run services delete gcp-gb
gcloud artifacts docker images delete
us-west1-docker.pkg.dev/cloud-calizo-calizo/guestbook-repo/gcp_gb



The commands show that they are completed successfully, removing the Cloud Run service and deleting the stored image. This final step concluded the lifecycle of the Cloud Run Guestbook deployment, ensuring a clean and cost-efficient workspace.

Lab 6.4g Cloud Functions, PubSub

Step 1. Cloud Functions image blurring

This lab focused on building a Cloud Function that automatically analyzes uploaded images for violent or adult content using Google's Vision API, and if detected, blurs the image using ImageMagick before saving it to an output bucket. I began by cloning Google's sample repository and navigating into the imagemagick function directory. From there, two globally unique Cloud Storage buckets were created - one as the input bucket for uploads and another as the output destination for blurred results. Both were successfully provisioned in the us-west1 region, confirming the foundational setup for the function's workflow.

Step 2. Services setup

To support automated image classification and cloud-triggered execution, several APIs were enabled: vision.googleapis.com, cloudfunctions.googleapis.com, cloudbuild.googleapis.com, pubsub.googleapis.com, and eventarc.googleapis.com. Each API activation completed without errors, ensuring the environment had the proper dependencies for event-driven functions, build pipelines, and storage triggers.

Step 3. Code

In main.py, the logic of the Cloud Function was reviewed. The script imports the required clients from Google's SDK (storage and vision), as well as the Image class from the wand.image library to perform the image transformations. The blur_offensive_images function is triggered by storage events and determines whether the uploaded image is inappropriate using the Vision API's SafeSearch Detection feature. If the content is flagged with a rating of 5 for either adult or violence, the function invokes __blur_image() to apply Gaussian blur to the file and re-uploads it to the output bucket.

Step 4. Code (2)

After examining the helper function __blur_image, its process became clear:

1. After downloading the file from the bucket, where is it stored?

It is stored as a temporary file within the function's local runtime directory, created through Python's tempfile library.

2. What class in the ImageMagick package is used to do the blurring of the file?

The Image class from wand.image handles both the file load and blur operation.

3. What lines of code perform the blurring of the image and its storage back into the filesystem?

The blurring occurs within:

```
with Image(filename=temp_local_filename) as image:
    image.blur(radius=32, sigma=16)
    image.save(filename=temp_local_filename)
```

Afterward, the processed file is uploaded back to the Cloud Storage output bucket via:

```
new_blob.upload_from_filename(temp_local_filename)
```

The function then deletes the temporary file to prevent local buildup within the function environment.

Step 5. Set up service account

To ensure the function could read and write to both buckets and manage trigger events, appropriate permissions were granted to the App Engine default service account. The roles/storage.objectAdmin, roles/artifactregistry.reader, and roles/eventarc.eventReceiver roles were applied. Additionally, the Cloud Storage service agent was retrieved and given the roles/pubsub.publisher role, allowing the bucket events to publish to Pub/Sub topics that trigger the function. The policy bindings completed successfully, confirming that the service account possessed all the necessary scopes to execute and trigger the workflow end-to-end.

Step 6. Deploy the function

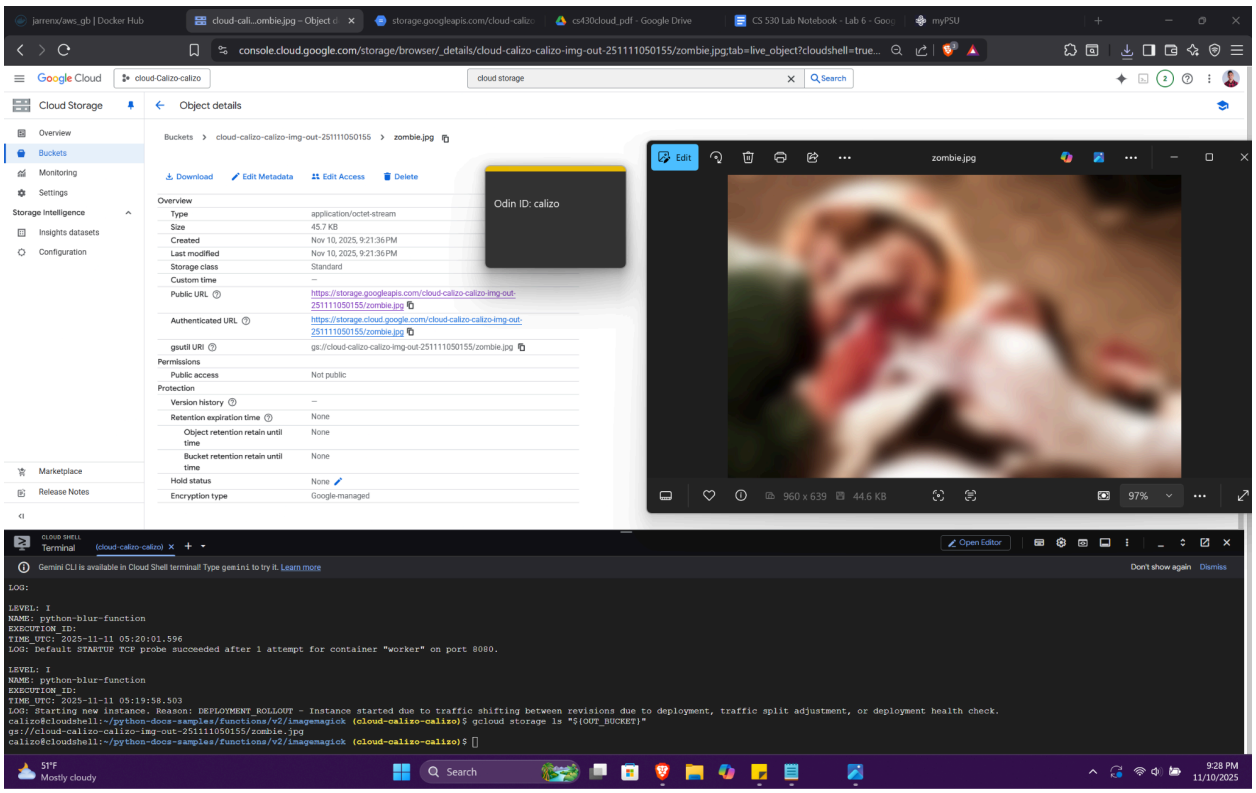
The Cloud Function was deployed using Python 3.11 under the name python-blur-function, triggered by new uploads to the input bucket. The environment variable BLURRED_BUCKET_NAME was set to direct the blurred results to the output bucket.

Deployment logs in the console confirmed that the IAM policy was set, the build was executed via Cloud Build, and the function endpoint was created in us-west1. The revision completed successfully, indicating that the trigger binding between EventArc, Pub/Sub, and Cloud Storage was fully functional.

Step 7. Test function

To verify functionality, I uploaded the provided zombie.jpg image to the input bucket. Within seconds, the Cloud Function triggered automatically and processed the image.

1. Take a screenshot of the blurred image in the output bucket for your lab notebook



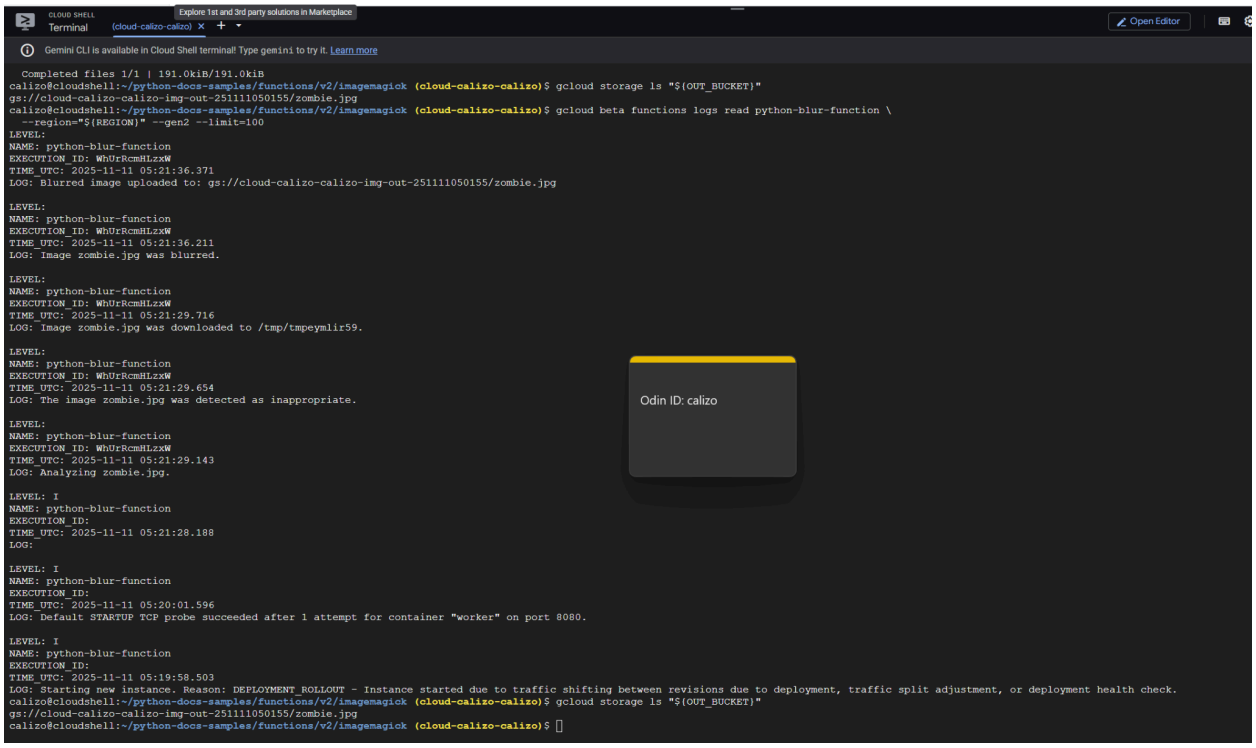
This shows the blurred result (zombie.jpg) in the output bucket, confirming the Vision API classified it as inappropriate and ImageMagick successfully applied the Gaussian blur.

The blurred image appeared in the output bucket at 44.6 KB, visually confirming that the image manipulation completed successfully.

To further verify execution, I inspected the logs with:

gcloud beta functions logs read python-blur-function --region=us-west1 --gen2 --limit=100

2. Include a screenshot of the output logs that show that the above image was blurred.



The logs displays the full detection and blur sequence:

- The image was downloaded to /tmp/tmpwylir59.jpg
- Vision API flagged the file as inappropriate
- The blur operation completed and the blurred image was uploaded to the output bucket

Step 8. Clean up

After validating successful operation, all created resources were deleted to maintain a clean environment. Using Cloud Shell, I removed the Cloud Function and both storage buckets with:

gcloud functions delete python-blur-function --gen2 --region=us-west1
gcloud storage rm -r gs://<UNIQUE_BUCKET_INPUT>
gcloud storage rm -r gs://<UNIQUE_BUCKET_OUTPUT>

Insight
This lab illustrated the seamless synergy between Cloud Functions, Pub/Sub, and Vision AI - a demonstration of event-driven automation that merges machine learning with serverless infrastructure. Watching an uploaded image autonomously classified, blurred, and stored encapsulated the power of GCP's ecosystem: scalable, reactive, and secure. It reaffirmed how modular cloud services can transform a simple function into an intelligent automation pipeline.

Step 9. Pub/Sub

This section introduced the publish-subscribe architecture - a decoupled messaging system where publishers broadcast messages to a topic and subscribers listen through subscriptions. In practice, this creates a scalable communication backbone between distributed services. Google Cloud Pub/Sub supports both push and pull delivery modes, and in this lab, I implemented both through the CLI and a Python-based programmatic interface.

Step 10. Create VM for subscriber

To simulate a subscriber environment, I created a dedicated service account named pubsub-lab and assigned it the Pub/Sub Editor role for publish and subscription permissions. Using this account, I launched a Compute Engine VM instance named pubsub with the Ubuntu 22.04 LTS image in us-west1-b. Once provisioned, I connected to the instance via SSH.

This setup allowed the Cloud Shell environment to act as the publisher while the VM functioned as the subscriber, creating a controlled test for message exchange through Cloud Pub/Sub.

Step 11. PubSub via CLI

In Cloud Shell, I created a new topic named topic-calizo using the command:

- gcloud pubsub topics create topic-$USER

A message was published to the topic using:

- gcloud pubsub topics publish projects/cloud-calizo-calizo/topics/topic-calizo --message="Message #1"

Then, on the VM, I created a pull-based subscription called sub-calizo linked to the same topic:

- gcloud pubsub subscriptions create sub-$USER --topic=projects/cloud-calizo-calizo/topics/topic-$USER

Pulling messages immediately after creation returned an empty list.

1. Why are there no items returned?

Because subscriptions only receive messages published after they are created. Since the first message was published before the subscription existed, it was not retained for delivery.

This verified that Cloud Pub/Sub does not retroactively deliver past messages without message retention or replay configured.

Step 12. PubSub via CLI (2)

Returning to Cloud Shell, I published a second message:

- gcloud pubsub topics publish projects/cloud-calizo-calizo/topics/topic-calizo --message="Message #2"

The operation successfully returned a new message ID.

1. What is the messageId of the published message?

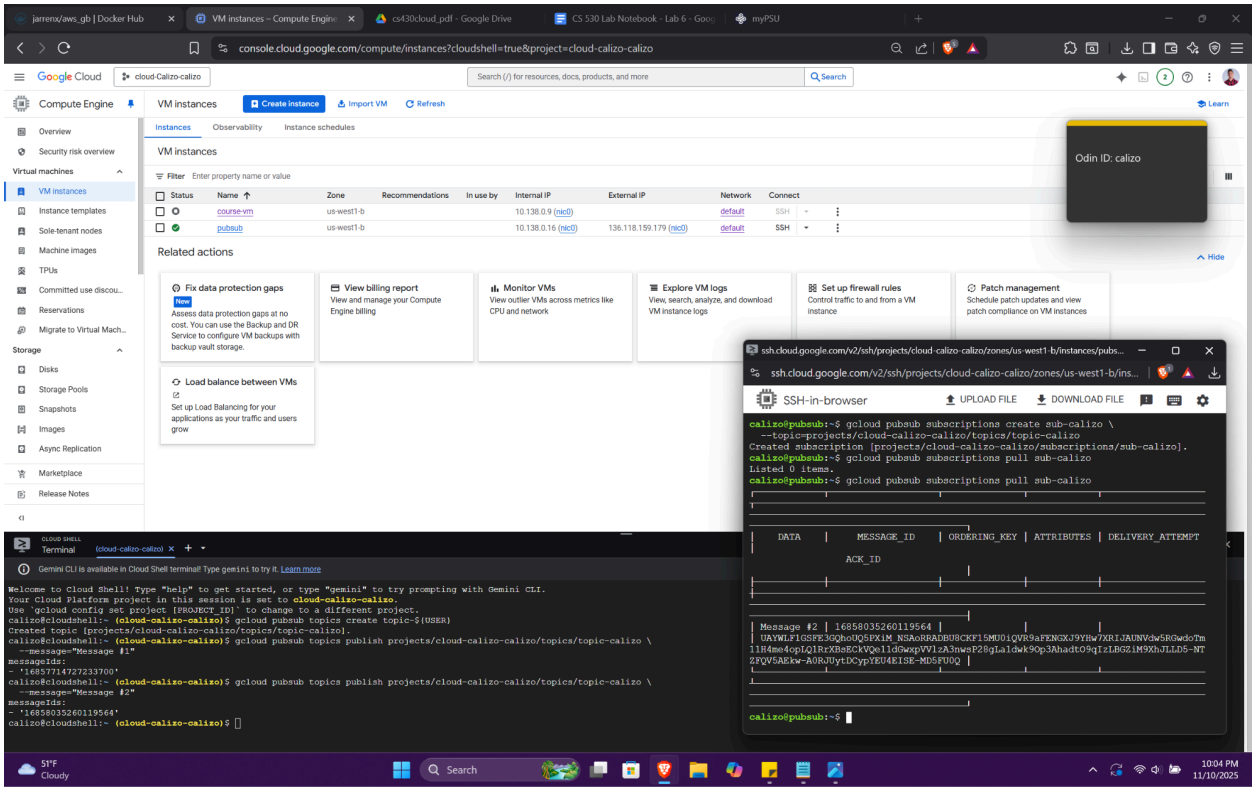
- 16858035260119564

Back on the VM, I pulled from the subscription again:

- gcloud pubsub subscriptions pull sub-calizo

This time, the message appeared successfully in the terminal, displaying the message data and the same message ID.

2. Take a screenshot of the output of the successful pull that includes the message and its messageId.



This confirms the pull command succeeded, showing "Message #2" and its unique messageId 16858035260119564 retrieved by the VM subscriber.

Step 13-14. Pub/Sub via Python

While the CLI demonstration confirmed Pub/Sub's core behavior, the next portion involved creating programmatic publisher and subscriber implementations in Python using the google-cloud-pubsub client library.

Both Cloud Shell and the VM were configured with Python virtual environments, activated via:

```
virtualenv -p python3 env
source env/bin/activate
pip install google-cloud-pubsub
```

In Cloud Shell, I created publisher.py, which initializes a topic called my_topic, accepts user input, appends a timestamp, and publishes the message asynchronously with a returned messageId.

On the VM, subscriber.py was created to connect to my_topic, create a subscription named my_subscription, and register a callback function that prints and acknowledges incoming messages in real time.

This setup mirrored real-world message pipelines, with Cloud Shell acting as the publishing producer and the VM serving as a continuously listening consumer.

Step 15. Test programs and clean up

I first executed the publisher program in Cloud Shell:

- python3 publisher.py

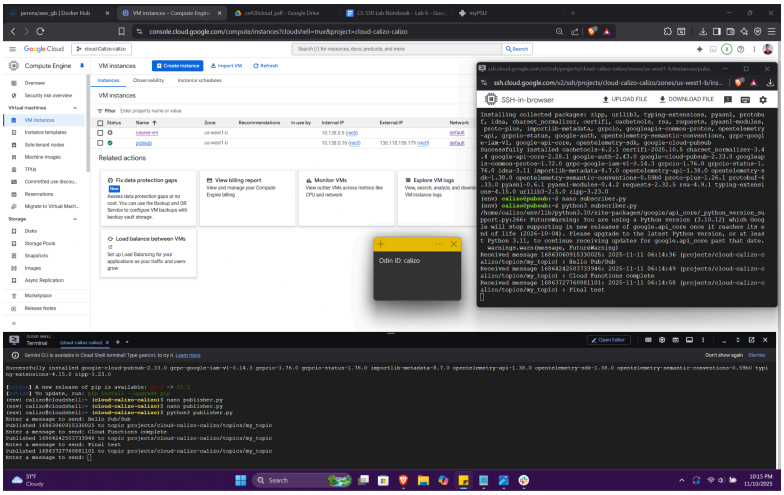
Then, I ran the subscriber on the VM:

- python3 subscriber.py

Upon sending messages such as "Hello Pub/Sub," "Cloud Functions complete," and "Final test," each was received by the subscriber in near real-time (Figure 3).

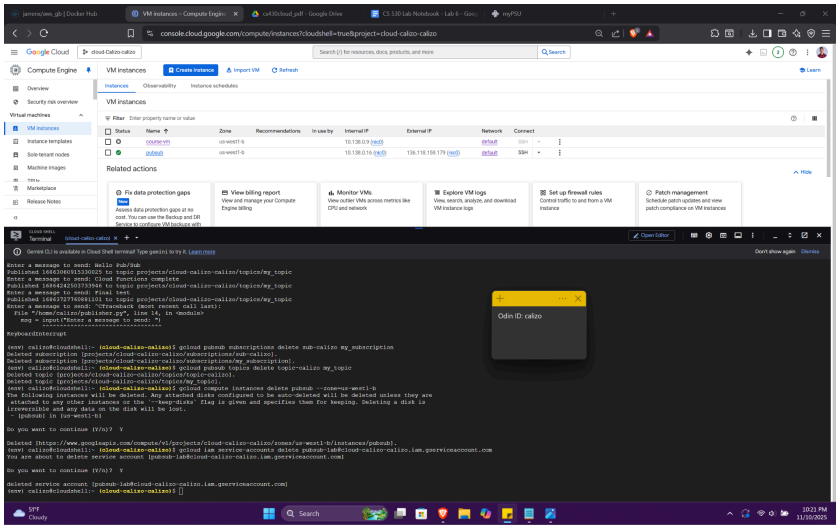
The timestamps, topic paths, and identical messageIds verified that Pub/Sub delivered messages exactly once and in order across environments.

1. Take a screenshot showing the messageIds and messages sent



Publisher program in Cloud Shell displaying multiple messages sent with their messageIds.

2. Take a screenshot showing the same messageIds and messages received



Subscriber program in VM displaying matching messageIds and contents for all messages received.

Insight

This lab elegantly tied together the full lifecycle of Pub/Sub messaging - from the creation of topics and subscriptions to the asynchronous communication between two distinct compute environments. Implementing the publisher and subscriber in Python highlighted the ease of integrating Pub/Sub into cloud applications, while the CLI reinforced a hands-on understanding of how messages propagate.

Through this exercise, I saw firsthand how Pub/Sub embodies the philosophy of decoupled communication - resilient, scalable, and ideal for microservice orchestration. It was a satisfying conclusion to see a message typed in one environment instantly materialize in another, linked by a bridge of managed cloud infrastructure.